

Next-token Preference: Scaling Unsupervised Reward Modeling

Jingxuan Fan¹, Yueying Li², Zhenting Qi¹, Dinghuai Zhang³,
Kianté Brantley^{1,4}, Sham M. Kakade^{1,4}, and Hanlin Zhang¹

¹Harvard University, ²Cornell University, ³Microsoft Research, ⁴Kempner Institute

Abstract

Learning from feedback advances the capabilities and safety of frontier models, but preference supervision remains costly and difficult to scale. We introduce next-token preference (NTP), an unsupervised reward-modeling method that constructs preference pairs from the continuation structure of raw web text. NTP uses each prefix’s original continuation as the chosen response and nearby continuations as in-batch negatives, without candidate generation or human or model-generated preference annotations. Training on 11M tokens of math-focused web data yields steady gains on RewardBench and RewardBench 2, with gains across model families and sizes. Across mathematical and general-domain web corpora, our method improves overall RewardBench 2 accuracy by 7.6–8.3 percentage points over initialization under the same token budget. Training on math-focused text yields gains of up to +16.1 on in-domain math subsets and consistent improvements on out-of-domain safety and general subsets. These reward models improve best-of- N selection on math, code, safety, and instruction following, and support math policy optimization competitive with supervised reward models of similar size. NTP also improves the label efficiency of subsequent supervised fine-tuning. With half of the labeled RLHF $\text{M}\times 1$ data, NTP initialization surpasses base initialization trained on the full dataset. These results suggest that web text contains transferable preference signals that can reduce dependence on costly preference annotations.

1 Introduction

Reinforcement learning from human feedback (RLHF) [10] has been the workhorse for building helpful and harmless language models [43, 3]. Reward models support this goal by guiding response selection and policy optimization. Their effectiveness depends on the preference data used to train them, yet obtaining these data at scale remains costly. Conventional pipelines require candidate generation and preference judgments from human annotators or models [13, 3]; automating judgments still incurs generation and scoring costs (Appendix D.1). Moreover, human feedback can inherently be noisy in part due to annotator subjectivity, inconsistency, and labeling errors, a phenomenon widely observed in preference-based learning [10, 43, 7, 59, 25]. Such systematic noise can significantly misguide reward models and produce serious misalignment downstream [32, 39]. Furthermore, as model capabilities scale, we face aligning systems whose relevant behaviors and failure modes exceed what humans (and comparably weak models) can reliably judge, making human-labeled reward signals fundamentally limited [5, 20]. These constraints motivate sources of reward-model supervision that do not require an explicit judgment for every comparison.

To address this supervision bottleneck, we introduce next-token preference (NTP), an unsupervised reward-modeling method that constructs preference pairs from the continuation structure of raw

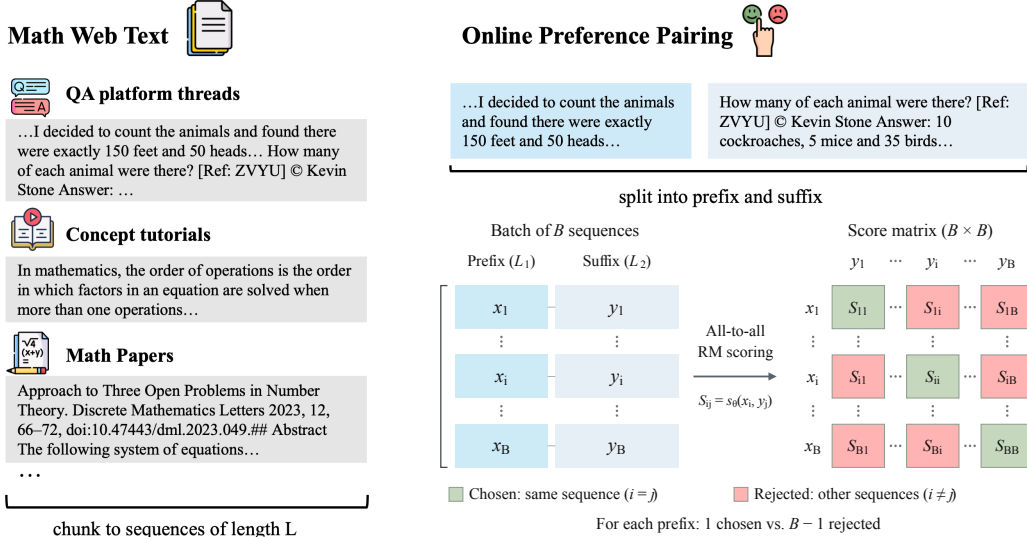


Figure 1: Schematic overview of our NTP reward model training workflow from web math text. Raw documents are split into prefix–suffix pairs, where the true continuation is treated as the chosen response and other in-batch continuations serve as implicit negatives. The reward model is trained with a Bradley–Terry objective over these online preference pairs, enabling scalable reward learning without human annotations.

web text, without human annotations or model-generated judgments. We split each text chunk into a prefix and continuation, using the prefix as the prompt and its original continuation as the chosen response (Fig. 1 and Section 2). Continuations from nearby chunks serve as rejected responses through in-batch pairings [27, 9]. The observed relationship between a prefix and its continuation thus supplies an implicit preference signal. Nearby continuations can share topic, vocabulary, and style while breaking the dependencies that connect a prefix to its original continuation. Comparing them encourages the reward model to learn semantic and logical consistency across the prefix–continuation boundary. All comparisons are assembled online from existing text, allowing preference supervision to scale with the corpus without candidate generation or preference annotation.

We use this construction to investigate whether the structure of web text can supply the supervision needed to learn useful, generalizable reward functions. Our main contributions are:

- **Transferable reward learning from web text.** Continuation consistency supports broader judgments of response quality: models trained on math text improve both reasoning and out-of-domain safety without labels for these behaviors. Gains grow with training data and extend across model families and sizes. Across two mathematical corpora and general-domain *FineWeb-Edu*, only 11M tokens per corpus yield peak overall RewardBench 2 gains of 7.6–8.3 percentage points over initialization [40].
- **Downstream utility.** The learned rewards guide improvements in language-model behavior through both response selection and policy optimization. They improve best-of- N selection on math, code, safety, and instruction following, and support mathematical policy optimization competitive with supervised reward models of comparable size.
- **A foundation for label-efficient reward modeling.** NTP also complements curated preference supervision by providing an effective mid-training initialization. With only half of the labeled

RLHFMix1 data, NTP initialization surpasses base initialization trained on the full dataset, supporting unsupervised reward learning as a scalable precursor to supervised training.

2 Continuation-Based Preference-Pair Construction

A reward model assigns a scalar score $s_\theta(x, y)$ to a response y conditioned on a prompt x and is trained to rank a preferred response y^+ above a rejected response y^- . Standard pipelines require human-written or model-generated prompts and candidate responses, together with preference annotations, all of which incur substantial costs at scale (see [Appendix D.1](#)). We introduce next-token preference (NTP), an unsupervised method that trains reward models directly from raw web text without relying on curated preference pairs. NTP uses a text prefix as the prompt and its original suffix as the chosen response. Rejected responses are suffixes from nearby text chunks that share the surrounding context but do not directly follow that prefix. We form these comparisons by grouping nearby chunks into a batch and pairing each prefix with the other chunks’ suffixes. This provides preference supervision without generating new responses or collecting preference judgments.

In our main experiments, we construct comparisons from *FineMath-4plus* [2], a collection of mathematical web documents filtered from CommonCrawl. We additionally use *MegaMath-Web-Pro-Max* [56], another mathematical web corpus, and *FineWeb-Edu* [44], a general-domain educational web corpus, to test generalization to other math and non-math web corpora (contamination checks in [Appendix E.2](#)).

For each corpus, we concatenate documents into a continuous token stream and divide it into fixed-length blocks of $L = L_1 + L_2$ tokens. Each block is split into an L_1 -token prefix x and an L_2 -token continuation y , forming a prompt–response pair (x, y) ([Fig. 1](#)). The split need not coincide with a sentence boundary (dataset parameters and sensitivity to split position in [Appendix E.1](#)).

Grouping B nearby chunks from this stream gives a batch of prefix–continuation pairs $\{(x_i, y_i)\}_{i=1}^B$. For each prefix x_i , we set $y_i^+ = y_i$ and $y_{ij}^- = y_j$ for every $j \neq i$. Each prompt therefore yields $B - 1$ comparisons of the form

$$(x_i, y_i^+, y_{ij}^-) = (x_i, y_i, y_j), \quad j \neq i.$$

This all-to-all construction yields $B(B - 1)$ preference comparisons per batch, assembled online during training. Suffixes from nearby chunks of the same mathematical article can share topic, vocabulary, and style while breaking the dependencies that connect a prefix to its true suffix. Their shared context limits shortcuts based on topic or style and encourages the reward model to learn semantic and logical dependencies across the prefix–suffix boundary. Negative difficulty depends

Algorithm 1: NTP: Reward Model Training from Web Data

```
# split(): breakpoint sampler on a continuous
# text sequence
# RM(): reward model that outputs a scalar
# score for (prompt, response)
for seq_batch in data_loader:
    prompts, responses = split(seq_batch)
    # S[i, j] = RM(x_i, y_j)
    score_mat = RM(prompts, responses)
    # S, [B, B]
    pos_scores = diag(score_mat)
    # s_pos, [B] (self-pairs = chosen)
    neg_scores = offdiag(score_mat)
    # s_neg, [B, B-1] (cross-pairs = rejected)
    bt_loss = BT(pos_scores, neg_scores)
    c_loss = CENTER(pos_scores, neg_scores)
    loss = bt_loss + c_loss
    loss.backward()
    update(RM.param)

def BT(s_pos, s_neg):
    # Bradley-Terry loss
    logits = s_pos[:, None] - s_neg
    return -log_sigmoid(logits).mean()

def CENTER(s_pos, s_neg):
    # score-centering regularizer
    return centering_coeff * (s_pos**2 +
                               s_neg**2).mean()
```

on batch composition and splitting (Section 3.2 and Appendix E.3). Our experiments on each representative corpus use only 11M tokens, a small fraction of the trillions of tokens available in web corpora [44]. Applying the same construction to additional web text expands the supply of preference comparisons without candidate generation or preference annotation.

We train the reward model on these comparisons with the Bradley–Terry objective [6], averaging the negative log-likelihood over all $B(B - 1)$ in-batch comparisons:

$$\mathcal{L}_{\text{BT}} = -\frac{1}{B(B-1)} \sum_{i=1}^B \sum_{j \neq i} \log \sigma(s_{\theta}(x_i, y_i^+) - s_{\theta}(x_i, y_{ij}^-)). \quad (1)$$

To stabilize training, we add score-centering regularization to the Bradley–Terry objective [19]. This is motivated by two properties of Bradley–Terry training: (i) the preference likelihood depends only on score differences and is therefore underdetermined up to a prompt-dependent offset, allowing the absolute reward scale to drift; and (ii) unconstrained optimization can inflate score magnitudes to produce overconfident margins that amplify spurious correlations and induce heavy-tailed reward distributions that are harmful for downstream selection. We therefore penalize large-magnitude reward outputs, encouraging a well-behaved and comparable score scale across prompts:

$$\mathcal{L}_{\text{center}} = \mathbb{E} \left[s_{\theta}(x_i, y_i^+)^2 + \frac{1}{B-1} \sum_{j \neq i} s_{\theta}(x_i, y_{ij}^-)^2 \right]. \quad (2)$$

Our final RM training loss is

$$\mathcal{L} = \mathcal{L}_{\text{BT}} + c \mathcal{L}_{\text{center}}, \quad (3)$$

where c is the centering coefficient. Constraining the reward range reduces overfitting to corpus-specific artifacts and improves robustness across diverse prompts and continuations (Algorithm 1).

3 Results

3.1 Reward-Model Training and Preference-Alignment Evaluation

We evaluate NTP by training reward models from four backbones spanning two model families and parameter scales from 1B to 7B: base Llama-3.2-1B and Llama-3.2-3B [41], and instruction-tuned Qwen2.5-3B-Instruct and Qwen2.5-7B-Instruct [45] (recipe and hyperparameters in Appendix E.5 and Table 9). All four models are trained on the web-text comparisons constructed in Section 2 using objective in Eq. (3), under the same 11M-token budget. We use Llama-3.2-3B as the representative backbone for controlled ablations.

We first evaluate the trained reward models’ general preference alignment. We use RewardBench [33] and RewardBench 2 [40], two benchmarks that measure whether reward models rank preferred responses above rejected alternatives across several dimensions of response quality. RewardBench covers response helpfulness and instruction following (Chat, Chat Hard), refusal of harmful requests while answering benign ones (Safety), and mathematical and coding correctness (Reasoning). RewardBench 2 covers mathematical correctness (Math), appropriate responses to harmful and benign but potentially risky requests (Safety), sensitivity to factual errors (Factuality), reliable ranking of responses with similar quality (Ties), relevance to the requested task (Focus), and compliance with verifiable instructions (Precise IF). Because our primary corpus is mathematical, we report *in-domain* (ID) performance (Reasoning subset for RewardBench; Math subset for RewardBench 2), *out-of-domain* (OOD) performance on safety (Safety subsets for both) and other remaining subsets, alongside the overall benchmark average.

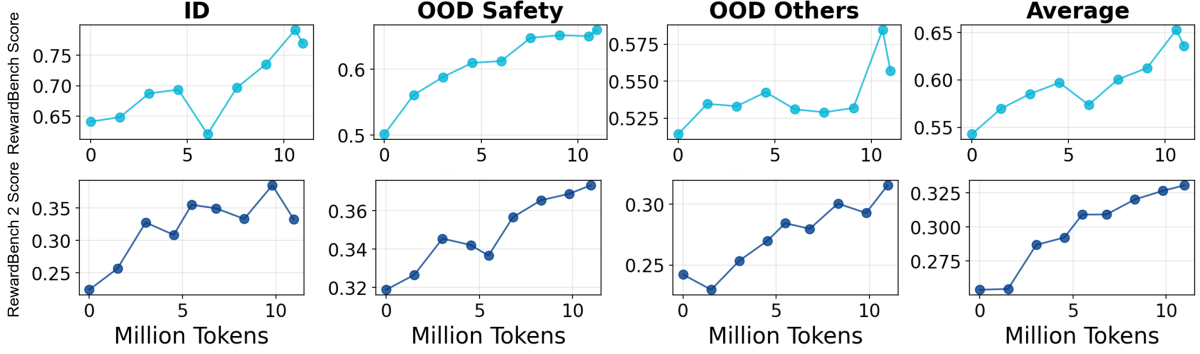


Figure 2: Scalability of NTP with respect to data size. Reward models initialized from pretrained Llama-3.2-3B improve across RewardBench and RewardBench 2 subsets as the token budget increases to 11M.

For Llama-3.2-3B, RewardBench and RewardBench 2 accuracy improves steadily over the 11M-token budget (Fig. 2 and Appendix H.2), suggesting that large-scale raw text provides a useful preference-learning signal even without curated comparisons. At 11M tokens, the gains extend beyond mathematical reasoning to general preference alignment on both benchmarks. On RewardBench, accuracy in judging appropriate refusal improves by 16.0 percentage points over initialization, exceeding the 13.1-point gain in evaluating mathematical and coding correctness. Accuracy in identifying better responses in challenging conversations also improves by 7.0 points. RewardBench 2 likewise shows substantial improvements in judging response safety and relevance to the user’s request (Table 14). To understand this surprising transfer from mathematical text to safety, we examine performance on individual questions in the RewardBench 2 *Safety* subset. The analysis indicates that the gains extend beyond learning superficial refusal cues (Appendix H.4). Thus, although training only contrasts consistent mathematical continuations with mismatched ones, the learned reward function transfers to judgments of appropriate refusal, response helpfulness, and task relevance without explicit preference supervision for these behaviors.

Using RewardBench 2, we also compare against training the same Llama-3.2-3B backbone under a matched 11M-token budget on Skywork-80K-v0.2, a curated dataset of approximately 77K labeled preference pairs [35], and RLHFMix1, a curated mixture of 244K preference pairs labeled by humans and GPT-4 [16]. At the same token budget, NTP improves overall accuracy by 7.7 percentage points, compared with 1.8 for Skywork and 0.8 for RLHFMix1 (Table 1). This gain also approaches those from training on the full curated datasets (9.8 and 8.3 points, respectively), despite using raw web text without human or LLM preference labels. Dataset construction and equivalent pair matching are detailed in Appendix F.1.

Table 1: Overall RewardBench 2 gains (percentage points) over Llama-3.2-3B initialization. **Bold**: best gain at the matched 11M-token budget.

Method	Budget	RewardBench 2 gain
FineMath NTP (ours)	11M tokens	+7.7
Skywork-80K-v0.2	≈9K pairs (11M)	+1.8
Skywork-80K-v0.2	77K pairs (full)	+9.8
RLHFMix1	≈22K pairs (11M)	+0.8
RLHFMix1	244K pairs (full)	+8.3

Table 2: Average RewardBench and RewardBench 2 scores and leaderboard ranks for RMs trained with different initialization backbones.

Backbone	RewardBench		RewardBench 2	
	Score	Rank	Score	Rank
Llama-3.2-1B	60.0	3	33.2	3
Llama-3.2-3B	65.6	4	36.2	9
Qwen2.5-3B-Inst.	70.0	3	46.2	8
Qwen2.5-7B-Inst.	73.8	18	57.0	5

lies, scales, and base and instruction-tuned initializations.

3.2 Ablation Studies

Using Llama-3.2-3B and the default training setup above, we ablate batch size, data splitting, and score centering loss. We also compare against continued pretraining on the same text to test whether corpus exposure alone explains the gains.

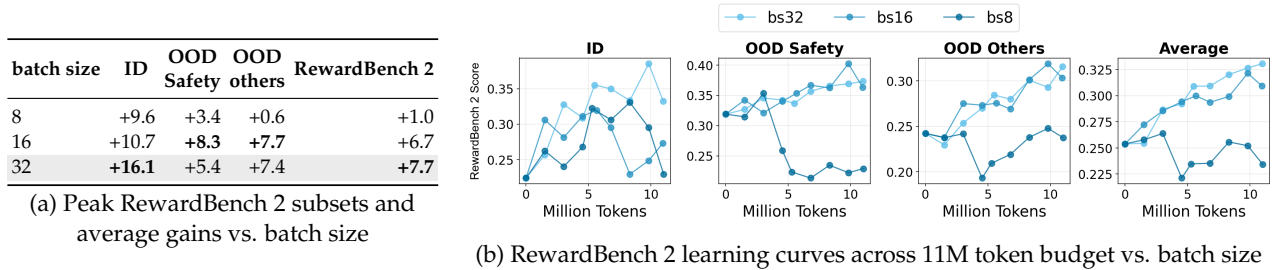


Figure 3: Effect of batch size on peak gains and learning trajectory of RewardBench 2.

Batch size. Increasing batch size in our scheme consistently improves RM performance. Each batch of B prefix-suffix pairs provides B positives and $B(B - 1)$ cross-pair negatives, so scaling B increases ranking supervision per update nearly quadratically. Accordingly, larger batches yield higher peak gains over the initialized seed on RewardBench 2 (Fig. 3a): ID improvements scale cleanly with B , while OOD gains plateau with little benefit beyond $B=16$. Larger B also reaches higher accuracy with a more stable trajectory over the 11M-token budget (Fig. 3b), with ID stability and final performance continuing to improve even as OOD curves largely saturate after $B=16$.

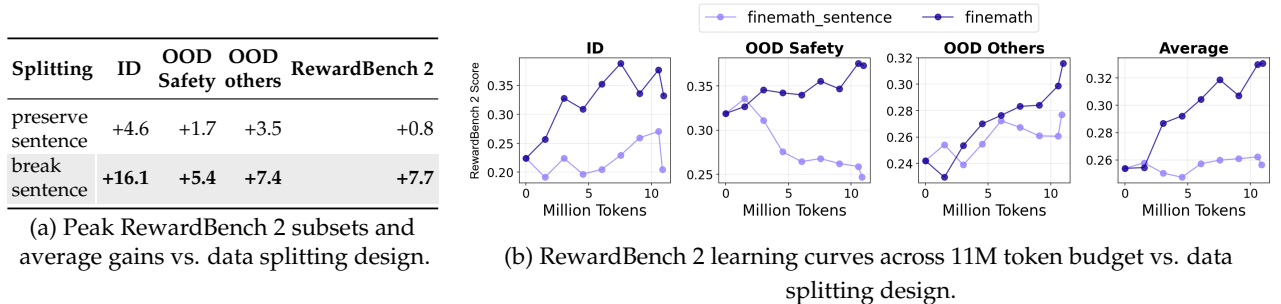


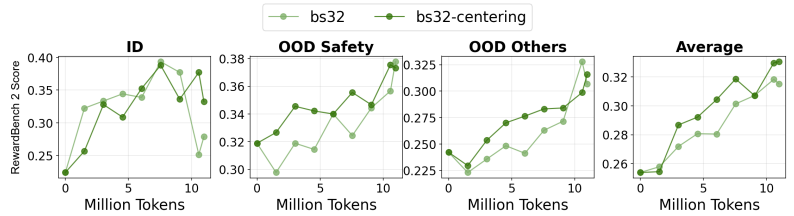
Figure 4: Effect of data splitting design on peak gains and learning trajectory of RewardBench 2.

Data splitting format. Our online preference data depends on how we split raw web text into prefix-suffix pairs: either only at sentence boundaries (*preserve sentence*) or potentially mid-sentence (*break*

sentence), which changes the difficulty of the induced in-batch negative samples. Allowing sentence breaks yields much larger gains over the initialized seed on RewardBench 2 and across all ID/OOD subsets (Fig. 4a,b). We attribute this to harder negative examples: in the *preserve sentence* setting, packing prefix-suffix sequences while enforcing boundary constraints inevitably discards many candidate spans, leaving resulting batches with examples from disparate passages and making negatives less contextually similar and easier to reject. In contrast, when sentence breaks are allowed, many in-batch sequences remain contiguous chunks from the same underlying web text context, so cross-pair negatives tend to be more contextually confusable and require finer-grained semantic and contextual consistency to rank correctly. Appendix E.4 provides more details on the *preserve sentence* splitting algorithm and intuition of its failure.

Centering	ID	OOD Safety	OOD others	RewardBench 2	BoN
w/o centering	+16.9	+5.9	+8.6	+6.4	+2.8
w/ centering	+16.4	+5.7	+7.4	+7.7	+5.4

(a) Peak RewardBench 2 subsets, average gains and BoN max achieved performance (MAP) gain vs. centering loss design.



(b) RewardBench 2 learning curves across 11M token budget vs. centering loss.

Figure 5: Effect of centering loss on RewardBench 2 peak gains, learning trajectory and BoN MAP.

Centering loss. We ablate the centering regularizer in our continuation-based Bradley-Terry RM objective by training with and without the quadratic penalty $\mathcal{L}_{\text{center}}$ (Eq. (2)), which keeps chosen and in-batch rejected scores near zero and limits reward-scale drift on noisy web text. Training with centering yields a modest improvement in average RewardBench 2 performance, with peak gains over initialization of +7.7 versus +6.4 points without centering (Fig. 5a,b). Centering provides a larger benefit for downstream response selection, measured by best-of- N (BoN), which selects the highest-scoring response among N actor-generated candidates (Section 3.4). On GSM8K [12] with a Llama-3.1-8B-Instruct actor, centering substantially boosts the gain in Maximum Achieved Performance (ΔMAP) over initialization (+5.4 vs. +2.8, Fig. 5a). The benefit also grows with selection strength: the centered RM’s accuracy increases monotonically with N before saturating, while the uncentered RM peaks at small N and eventually falls below the random baseline (Fig. 6).

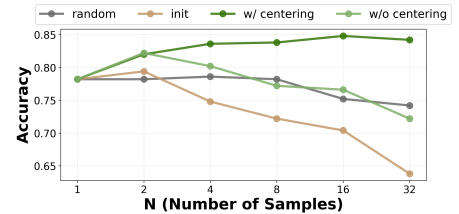


Figure 6: BoN accuracy curve on Llama-3.1-8B-Instruct GSM8K rollouts for RMs trained with or without centering loss (init-initialized seed).

Continued-pretraining control. To isolate the effect of the training objective, we continue pretraining Llama-3.2-3B with next-token prediction on the same 11M *FineMath* tokens in the same order as NTP training. Since this model has no reward head, we compare two readouts trained on 512 auxiliary preference pairs from *FineMath* blocks excluded from the earlier reward-model and continued-pretraining runs. Each pair uses a document prefix as the prompt, its original continuation as chosen, and a perturbed copy as rejected; evaluation uses RewardBench 2. The *frozen-backbone probe* fixes the backbone and fits a linear scoring head to the final-layer hidden state at the last response token. We fit it with the Bradley-Terry objective to score original continuations above perturbed copies. *Full reward-model conversion* jointly finetunes a newly initialized scalar reward head and all

backbone parameters on the same pairs for three epochs with the same objective. We apply both readouts to the continued-pretrained and untouched base models (Appendix E.6).

Continued pretraining on the same text does not improve reward-model performance over the untouched base model under either the frozen-backbone probe or full reward-model conversion (Table 3). Replacing the NTP reward model’s trained head with the same probe preserves its performance, indicating that the probe can recover reward signals learned by NTP. Together, these findings support the contribution of NTP preference learning beyond additional next-token training on the same corpus.

Table 3: RewardBench 2 scores for Llama-3.2-3B before training and after continued pretraining (CPT) or NTP reward-model training (RM), each on 11M FineMath tokens.

Readout	Untrained	CPT	RM
Frozen probe	30.0	28.9	36.4
Conversion (512 pairs)	32.3	31.2	–
Original RM head	–	–	36.2

3.3 Generalization to Other Web Corpora

To test whether NTP remains effective beyond *FineMath-4plus*, we train on *MegaMath-Web-Pro-Max* [56], another mathematical web corpus, and *FineWeb-Edu* [44], a general-domain educational web corpus. We use the same Llama-3.2-3B initialization and training settings as the *FineMath-4plus* run, with 11M tokens per corpus.

Table 4 reports peak average RewardBench 2 accuracy gains over initialization. *FineMath-4plus*, *MegaMath-Web-Pro-Max*, and *FineWeb-Edu* yield gains of 7.7, 8.3, and 7.6 percentage points, respectively. *MegaMath-Web-Pro-Max* gives the largest gain, exceeding *FineMath-4plus* by 0.6 points, while general-domain *FineWeb-Edu* comes within 0.1 points of *FineMath-4plus*. These results show that NTP’s effectiveness extends across web corpora and that substantial reward-model improvements can also be obtained from general-domain educational text.

Table 4: Peak average RewardBench 2 accuracy gains (percentage points) over the shared initialization for Llama-3.2-3B RMs trained on 11M tokens from each corpus.

Training corpus	RewardBench 2 gain
FineMath-4plus	+7.7
MegaMath-Web-Pro-Max	+8.3
FineWeb-Edu	+7.6

3.4 Best-of-N Accuracy and Scaling

Motivated by ID and OOD RewardBench and RewardBench 2 gains, we evaluate our web-trained RMs using BoN selection. We sample $N \in \{1, 2, 4, 8, 16, 32\}$ responses per prompt, select the highest-scoring one, and report Maximum Achieved Performance (MAP), the best task accuracy over N . We evaluate three actors, Llama-3.2-1B-Instruct, Llama-3.2-3B-Instruct, and Llama-3.1-8B-Instruct, on two in-domain mathematical reasoning tasks, MATH500 and GSM8K [34, 12], and three out-of-domain tasks: HumanEval+ for code generation, ToxiGen for safety, and IFEval for instruction following [8, 37, 28, 63]. We use two RMs trained with our online continuation-based method (Section 2): FineMath-RM-Llama-3.2-3B, initialized from Llama-3.2-3B, and FineMath-RM-Qwen-2.5-7B, initialized from Qwen2.5-7B-Instruct. We compare against three off-the-shelf supervised RMs: Skywork-Reward-V2-Llama-3.1-8B, Skywork-Reward-V2-Llama-3.2-3B [36], and PKU’s beaver-7b-unified-reward, a safety-oriented RM [14, 31], as well as random selection. When measuring gains over initialization, we also score the same candidate sets with each RM’s untrained initialization and report Δ MAP. Evaluation details appear in Appendix G.

Across ID and OOD tasks, BoN gains become more consistent as actor size grows. Accuracy generally improves with N : gains are weakest for FineMath-RM-Llama-3.2-3B with the smallest Llama-3.2-1B-Instruct actor, but become clear for larger actors. FineMath-RM-Qwen-2.5-7B yields larger, often monotonic gains, especially with stronger actors. These results suggest that our RMs reliably select better responses across ID math reasoning and OOD code, safety, and instruction following. Actor-size scaling benefits FineMath-RM-Llama-3.2-3B more on ID than OOD tasks, while FineMath-RM-Qwen-2.5-7B shows similar trends across tasks (Fig. 7).

Supervised RMs vary substantially in BoN performance. Across tasks and actors, the Beaver RM underperforms the stronger Skywork RM by 4 to 25 MAP points, showing that curated preference training alone does not guarantee a strong selection signal. Our RMs become more competitive with the strongest supervised baselines as actor size grows. Moreover, FineMath-RM-Qwen-2.5-7B exceeds Skywork-Reward-V2-Llama-3.2-3B on OOD ToxiGen at every actor size and matches Skywork-Reward-V2-Llama-3.1-8B on ID MATH500 with the 8B actor (Fig. 7). Compared with the Beaver RM, FineMath-RM-Qwen-2.5-7B achieves higher MAP on every task and actor size, while FineMath-RM-Llama-3.2-3B matches or exceeds it on GSM8K, MATH500, HumanEval+, and IFEval in all but one setting, with lower MAP mainly on ToxiGen. On IFEval, the Beaver RM performs worse than random selection as N grows, whereas both of our RMs improve over the actor. Although the strongest curated-data RMs remain ahead overall, our web-trained RMs outperform a supervised safety-oriented RM in most settings without human or LLM labels. These results indicate that raw math web text yields an effective selection signal that strengthens with RM and actor size.

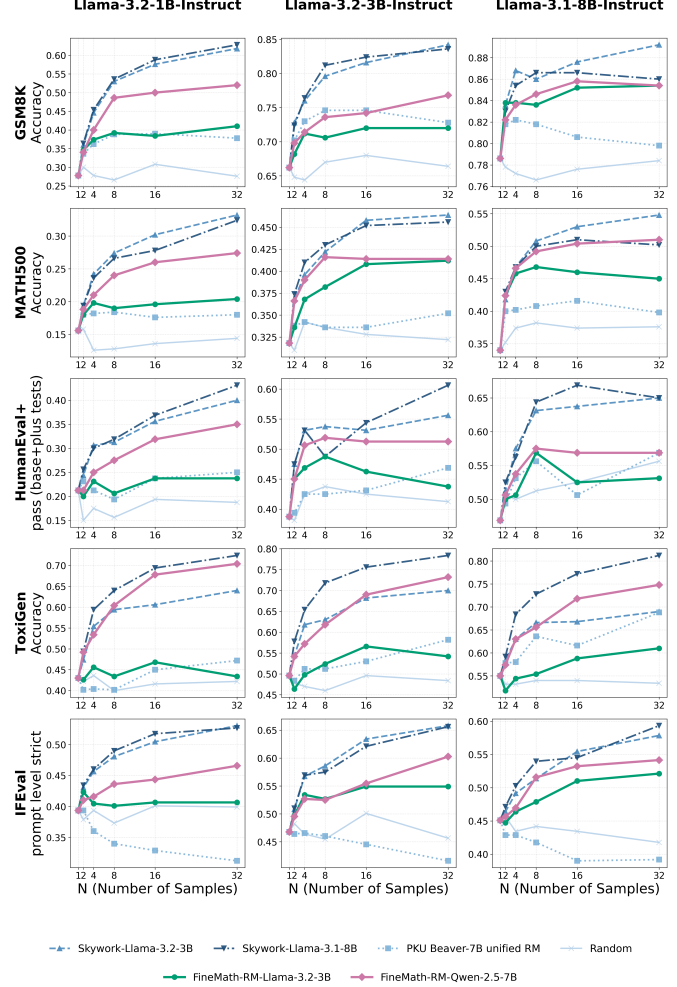


Figure 7: BoN scaling curves of RMs across two ID math tasks (GSM8K, MATH500) and three OOD tasks (HumanEval+, ToxiGen, IFEval).

3.5 Actor Training

Because `FineMath-RM-Qwen-2.5-7B` reliably selects stronger mathematical responses in BoN, we also test it as a reward for policy training. We train `Llama-3.2-3B-Instruct` and `Llama-3.1-8B-Instruct` with GRPO [48] on MATH and GSM8K for five epochs, comparing against the two Skywork RMs, `beaver-7b-unified-reward` [14, 31], and our RM’s untrained initialization. We report peak mean@1 test accuracy (Appendices I.1 and I.2).

Our RM improves both actors on both tasks, achieves the best or second-best peak accuracy in three of the four settings, and gives the best GSM8K result for the 8B actor (Table 5). It exceeds the Beaver RM by 0.4–2.0 percentage points across all four settings, showing that NTP training on raw web text provides an effective policy-training reward compared with this supervised baseline. It also outperforms its untrained initialization in every setting, supporting a consistent benefit from reward learning.

Table 5: Peak mean@1 test accuracy within five GRPO epochs. Both actors use instruction-tuned initializations. **Bold** and underlined values denote the best and second-best results, respectively.

	Actor: Llama-3.2- 3B-Instruct		Actor: Llama-3.1- 8B-Instruct	
	MATH	GSM8K	MATH	GSM8K
Before training	0.268	0.789	0.423	0.876
Skywork-Reward-V2-Llama-3.1-8B	0.419	0.833	<u>0.447</u>	0.882
Skywork-Reward-V2-Llama-3.2-3B	0.439	0.819	0.465	<u>0.884</u>
beaver-7b-unified-reward	0.416	0.803	0.430	0.881
FineMath-RM-Qwen-2.5-7B	<u>0.420</u>	<u>0.823</u>	0.437	0.886
FineMath-RM-Qwen-2.5-7B-Init	0.399	0.812	0.428	0.879

3.6 Mid-Training for Reward Modeling

Although NTP is competitive with supervised reward training under a matched token budget (Table 1), our models still in many cases underperform the strongest reward models trained on large curated preference datasets. We therefore also study NTP as a complement to supervised training, rather than a head-to-head comparison, and ask whether an initialization learned from unlabeled web text can reduce the need for curated preference labels.

We test whether the 11M-token FineMath RM checkpoint provides a useful initialization for subsequent curated preference learning. We compare three `Llama-3.2-3B` starting points: the base model, our NTP-trained RM, and the instruction-tuned model. Each receives the same subsequent training on `RLHFMix1`, a uniformly sampled mixture of UltraFeedback, HH-RLHF, SHP, and Summarize-from-Feedback [16, 13, 3, 21, 53]. We evaluate all resulting RMs on RewardBench 2, reporting overall, ID-Math, OOD-Safety, and OOD-Others accuracy using the split defined in Section 3.1; training details are in Appendix J.

As expected, instruct initialization outperforms base initialization, consistent with standard practice of starting RM training from an SFT model [43, 3]. Crucially, our RM checkpoint recovers much of this benefit: it achieves +10.4 average gain on RewardBench 2, matching or exceeding instruct on key subcategories—OOD-Safety (+24.4 vs. +22.6) and ID (+21.4 vs. +19.4)—while substantially outpacing the base model on ID (+14.5) (Table 6 and Fig. 12). This indicates that our unsupervised RM training provides a particularly strong foundation for learning in-distribution preferences from curated data. Gains on OOD Others are modest, likely due to forgetting induced by distribution shift from web text to curated data; in OOD Safety, a similar initial drop is recovered and ultimately surpassed (Fig. 12).

Table 6: RewardBench 2 gains from training on curated preference dataset `RLHFMix1` under different initializations.

Checkpoint	ID	OOD Safety	OOD others	RewardBench 2
Llama-3.2-3B	+14.5	+22.3	+1.6	+8.3
Llama-3.2-3B-Instruct	+19.4	+22.6	+7.8	+13.1
Finemath-RM-Llama-3.2-3B	+21.4	+24.4	+2.0	+10.4

To assess whether NTP initialization reduces the need for curated supervision, we vary the amount of RLHFMix1 data (Appendix J.1). NTP initialization outperforms base initialization at every evaluated budget. With 50% of the labeled data (approximately 122K pairs), it reaches an overall RewardBench 2 score of approximately 44, matching its own full-data result and exceeding the end performance of base initialization with all 244K pairs.

4 Related Work

Sources of reward supervision. Conventional reward models learn to rank candidate responses from human comparisons [10, 52, 43, 3]. The human effort required to collect these comparisons motivates replacing some human evaluations with LLM judgments [60, 18]. Constitutional AI [4], UltraFeedback [13], and Self-Rewarding Language Models [58] use such judgments as feedback, while Semi-Supervised Reward Modeling (SSRM) [29] extends a labeled seed set with high-confidence comparisons pseudo-labeled by a reward model. Although these methods reduce human labeling, generating candidate responses and running LLM judges still incur monetary costs, as illustrated by our estimates in Appendix D.1. LLM judgments also remain susceptible to position bias [50] and non-transitive preferences [57]. Other methods derive supervision from model behavior without explicit preference judgments. POLAR [17], for example, pretrains on model-generated responses, using whether they come from the same policy to construct training labels. Unsupervised Process Reward Models (uPRM) [22] derive supervision for reasoning-step evaluation from a pretrained LLM’s token probabilities. Both approaches therefore retain a dependence on model-generated responses or model-based assessments to supply their training signal. NTP instead constructs comparisons directly from existing web text, treating each prefix’s original continuation as preferred over mismatched continuations. The text itself thus supplies both responses and preference labels, avoiding the generation and judging costs of preference-data collection.

From text discrimination to quality scoring. NTP builds on discriminative pretraining, which extracts supervision by contrasting observed data with mismatched or corrupted alternatives. Quick Thoughts [38], BERT’s next-sentence prediction [15], and Contrastive Predictive Coding (CPC) [55] use context matching or contrastive prediction to learn representations, while ELECTRA [11] learns by detecting generator-replaced tokens. Where these tasks serve representation learning, NTP formulates continuation matching as Bradley–Terry preference learning and retains the response-level scalar scorer as a reward model. Closer to NTP’s scoring objective, BLEURT [47] pretrains a reference-based metric on perturbed text with automatic targets before fine-tuning on human ratings, while DEAM [26] learns dialogue-coherence scores from semantically manipulated examples. NTP instead forms its preference pairs by matching and mismatching existing prefixes and continuations, requiring neither text perturbation nor semantic parsing. The question is then whether learning this continuation relationship produces rewards that transfer to broader preference judgments and supervised reward training. Such transfer also needs to hold when the scores guide model behavior, since proxy rewards can be exploited under optimization [23, 51]. We therefore evaluate the learned rewards through best-of- N selection and policy optimization as well as preference benchmarks (Section 3).

5 Concluding Remarks

We introduced next-token preference (NTP), an unsupervised reward-modeling method that learns scalar rewards from web-text continuations without candidate generation or human or model-generated preference judgments. Although this supervision comes from continuation consistency, the learned rewards transfer to broader judgments of response quality, including reasoning and out-of-domain safety after training on mathematical text. Gains grow with training data and extend across model families and sizes. Across mathematical and general-domain corpora, 11M tokens per corpus yield peak RewardBench 2 gains of 7.6–8.3 percentage points over initialization. This transfer also has practical value, improving best-of- N selection in math, code, safety, and instruction following, and supporting mathematical policy optimization competitive with similarly sized supervised reward models.

NTP also complements supervised reward learning by improving how effectively models use curated preference data. With only half of the labeled $\text{RLHFM}_{\text{v1}} \times 1$ data, NTP initialization surpasses base initialization trained on the full dataset. This complementary benefit suggests that NTP can serve as a form of reward-model pretraining, learning a useful reward prior from raw text before refinement with supervised preferences. Scaling NTP training and combining it with targeted supervision are promising directions for further reducing dependence on expensive preference data.

Acknowledgments

SK acknowledges the support from the National Science Foundation Grant under award IIS 2229881; JF, KB, SK and HZ acknowledge the Chan Zuckerberg Initiative Foundation for establishing the Kempner Institute for the Study of Natural and Artificial Intelligence. YL acknowledges Anvil AI and GPU allocations through allocation CIS230253 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by U.S. National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296. JF is currently at Scale AI, but this work was conducted while at Harvard University.

References

- [1] Mayank Agarwal, Ibrahim Abdelaziz, Kinjal Basu, Merve Unuvar, Luis A. Lastras, Yara Rizk, and Pavan Kapanipathi. ToolRM: Outcome reward models for tool-calling large language models, 2025. URL <https://arxiv.org/abs/2509.11963>.
- [2] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgrén, Xuan-Son Nguyen, Clémentine Fourier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. Smollm2: When smol goes big – data-centric training of a small language model, 2025. URL <https://arxiv.org/abs/2502.02737>.
- [3] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.
- [4] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- [5] Samuel R. Bowman, Jeeyoon Hyun, Ethan Perez, Edwin Chen, Craig Pettit, Scott Heiner, Kamilė Lukošiušė, Amanda Askell, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Christopher Olah, Daniela Amodei, Dario Amodei, Dawn Drain, Dustin Li, Eli Tran-Johnson, Jackson Kernion, Jamie Kerr, Jared Mueller,

- Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Liane Lovitt, Nelson Elhage, Nicholas Schiefer, Nicholas Joseph, Noemí Mercado, Nova DasSarma, Robin Larson, Sam McCandlish, Sandipan Kundu, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Timothy Telleen-Lawton, Tom Brown, Tom Henighan, Tristan Hume, Yuntao Bai, Zac Hatfield-Dodds, Ben Mann, and Jared Kaplan. Measuring progress on scalable oversight for large language models, 2022. URL <https://arxiv.org/abs/2211.03540>.
- [6] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952. ISSN 00063444, 14643510. URL <http://www.jstor.org/stable/2334029>.
- [7] Stephen Casper, Xander Davies, Claudia Shi, Thomas Krendl Gilbert, Jérémy Scheurer, Javier Rando, Rachel Freedman, Tomasz Korbak, David Lindner, Pedro Freire, et al. Open problems and fundamental limitations of reinforcement learning from human feedback. *arXiv preprint arXiv:2307.15217*, 2023.
- [8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgens Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- [9] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pages 1597–1607. PmLR, 2020.
- [10] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- [11] Kevin Clark, Minh-Thang Luong, Quoc V. Le, and Christopher D. Manning. Electra: Pre-training text encoders as discriminators rather than generators, 2020. URL <https://arxiv.org/abs/2003.10555>.
- [12] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- [13] Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Bingxiang He, Wei Zhu, Yuan Ni, Guotong Xie, Ruobing Xie, Yankai Lin, Zhiyuan Liu, and Maosong Sun. Ultrafeedback: Boosting language models with scaled ai feedback, 2024. URL <https://arxiv.org/abs/2310.01377>.
- [14] Josef Dai, Xuehai Pan, Ruiyang Sun, Jiaming Ji, Xinbo Xu, Mickel Liu, Yizhou Wang, and Yaodong Yang. Safe RLHF: Safe reinforcement learning from human feedback. In *The Twelfth International Conference on Learning Representations*, 2024.
- [15] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/v1/N19-1423. URL <https://aclanthology.org/N19-1423/>.
- [16] Hanze Dong, Wei Xiong, Bo Pang, Haoxiang Wang, Han Zhao, Yingbo Zhou, Nan Jiang, Doyen Sahoo, Caiming Xiong, and Tong Zhang. RLHF workflow: From reward modeling to online RLHF. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=a13aYUU9eU>.
- [17] Shihan Dou, Shichun Liu, Yuming Yang, Yicheng Zou, Yunhua Zhou, Shuhao Xing, Chenhao Huang, Qiming Ge, Demin Song, Haijun Lv, Songyang Gao, Chengqi Lv, Enyu Zhou, Honglin Guo, Zhiheng Xi, Wenwei Zhang, Qipeng Guo, Qi Zhang, Xipeng Qiu, Xuanjing Huang, Tao Gui, and Kai Chen. Pre-trained policy discriminators are general reward models. In *Advances in Neural Information Processing Systems*, 2025. URL <https://arxiv.org/abs/2507.05197>.

- [18] Yann Dubois, Chen Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, Carlos Guestrin, Percy S Liang, and Tatsunori B Hashimoto. AlpacaFarm: A simulation framework for methods that learn from human feedback. *Advances in Neural Information Processing Systems*, 36:30039–30069, 2023.
- [19] Jacob Eisenstein, Chirag Nagpal, Alekh Agarwal, Ahmad Beirami, Alex D’Amour, DJ Dvijotham, Adam Fisch, Katherine Heller, Stephen Pfohl, Deepak Ramachandran, Peter Shaw, and Jonathan Berant. Helping or herding? reward model ensembles mitigate but do not eliminate reward hacking, 2024. URL <https://arxiv.org/abs/2312.09244>.
- [20] Joshua Engels, David D. Baek, Subhash Kantamneni, and Max Tegmark. Scaling laws for scalable oversight. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=ulj6RqH8nM>.
- [21] Kawin Ethayarajh, Yejin Choi, and Swabha Swayamdipta. Understanding dataset difficulty with $\mathcal{V}$ -usable information. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato, editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 5988–6008. PMLR, 17–23 Jul 2022.
- [22] Artyom Gadetsky, Maxim Kodryan, Siba Smarak Panigrahi, Hang Guo, and Maria Brbić. Unsupervised process reward models, 2026. URL <https://arxiv.org/abs/2605.10158>.
- [23] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. *arXiv preprint arXiv:2210.10760*, 2022.
- [24] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 07 2024. URL <https://zenodo.org/records/12608602>.
- [25] Yang Gao, Dana Alon, and Donald Metzler. Impact of preference noise on the alignment performance of generative language models. *arXiv preprint arXiv:2404.09824*, 2024.
- [26] Sarik Ghazarian, Nuan Wen, Aram Galstyan, and Nanyun Peng. DEAM: Dialogue coherence evaluation using AMR-based semantic manipulations. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 771–785. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.acl-long.57. URL <https://aclanthology.org/2022.acl-long.57/>.
- [27] Raia Hadsell, Sumit Chopra, and Yann LeCun. Dimensionality reduction by learning an invariant mapping. In *2006 IEEE computer society conference on computer vision and pattern recognition (CVPR’06)*, volume 2, pages 1735–1742. IEEE, 2006.
- [28] Thomas Hartvigsen, Saadia Gabriel, Hamid Palangi, Maarten Sap, Dipankar Ray, and Ece Kamar. Toxigen: A large-scale machine-generated dataset for adversarial and implicit hate speech detection, 2022. URL <https://arxiv.org/abs/2203.09509>.
- [29] Yifei He, Haoxiang Wang, Ziyang Jiang, Alexandros Papangelis, and Han Zhao. Semi-supervised reward modeling via iterative self-training. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 7365–7377, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.434. URL <https://aclanthology.org/2024.findings-emnlp.434/>.
- [30] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021. URL <https://openreview.net/forum?id=7Bywt2mQsCe>.
- [31] Jiaming Ji, Donghai Hong, Borong Zhang, Boyuan Chen, Juntao Dai, Boren Zheng, Tianyi Qiu, Jiayi Zhou, Kaile Wang, Boxuan Li, Sirui Han, Yike Guo, and Yaodong Yang. Pku-saferlhf: Towards multi-level safety alignment for llms with human preference, 2025. URL <https://arxiv.org/abs/2406.15513>.

- [32] Tomasz Korbak, Kejian Shi, Angelica Chen, Rasika Vinayak Bhalerao, Christopher Buckley, Jason Phang, Samuel R Bowman, and Ethan Perez. Pretraining language models with human preferences. In *International Conference on Machine Learning*, pages 17506–17533. PMLR, 2023.
- [33] Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, Noah A. Smith, and Hannaneh Hajishirzi. RewardBench: Evaluating reward models for language modeling, 2024. URL <https://arxiv.org/abs/2403.13787>.
- [34] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step, 2023. URL <https://arxiv.org/abs/2305.20050>.
- [35] Chris Yuhao Liu, Liang Zeng, Jiakai Liu, Rui Yan, Jujie He, Chaojie Wang, Shuicheng Yan, Yang Liu, and Yahui Zhou. Skywork-reward: Bag of tricks for reward modeling in llms, 2024. URL <https://arxiv.org/abs/2410.18451>.
- [36] Chris Yuhao Liu, Liang Zeng, Yuzhen Xiao, Jujie He, Jiakai Liu, Chaojie Wang, Rui Yan, Wei Shen, Fuxiang Zhang, Jiacheng Xu, Yang Liu, and Yahui Zhou. Skywork-reward-v2: Scaling preference data curation via human-ai synergy, 2025. URL <https://arxiv.org/abs/2507.01352>.
- [37] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023.
- [38] Lajanugen Logeswaran and Honglak Lee. An efficient framework for learning sentence representations. In *International Conference on Learning Representations*, 2018. URL <https://arxiv.org/abs/1803.02893>.
- [39] Monte MacDiarmid, Benjamin Wright, Jonathan Uesato, Joe Benton, Jon Kutasov, Sara Price, Naia Bouscal, Sam Bowman, Trenton Bricken, Alex Cloud, et al. Natural emergent misalignment from reward hacking in production rl. *arXiv preprint arXiv:2511.18397*, 2025.
- [40] Saumya Malik, Valentina Pyatkin, Sander Land, Jacob Morrison, Noah A. Smith, Hannaneh Hajishirzi, and Nathan Lambert. RewardBench 2: Advancing reward model evaluation, 2025. URL <https://arxiv.org/abs/2506.01937>.
- [41] Meta. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- [42] OpenAI, :, Aaron Hurst, Adam Lerer, Adam P. Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, Aleksander Madry, Alex Baker-Whitcomb, Alex Beutel, Alex Borzunov, Alex Carney, Alex Chow, Alex Kirillov, Alex Nichol, Alex Paino, Alex Renzin, Alex Tachard Passos, Alexander Kirillov, Alexi Christakis, Alexis Conneau, Ali Kamali, Allan Jabri, Allison Moyer, Allison Tam, Amadou Crookes, Amin Tootoochian, Amin Tootoonchian, Ananya Kumar, Andrea VALLONE, Andrej Karpathy, Andrew Braunstein, Andrew Cann, Andrew Codisputi, Andrew Galu, Andrew Kondrich, Andrew Tulloch, Andrey Mishchenko, Angela Baek, Angela Jiang, Antoine Pélisse, Antonia Woodford, Anuj Gosalia, Arka Dhar, Ashley Pantuliano, Avi Nayak, Avital Oliver, Barret Zoph, Behrooz Ghorbani, Ben Leimberger, Ben Rossen, Ben Sokolowsky, Ben Wang, Benjamin Zweig, Beth Hoover, Blake Samic, Bob McGrew, Bobby Spero, Bogó Giertler, Bowen Cheng, Brad Lightcap, Brandon Walkin, Brendan Quinn, Brian Guarraci, Brian Hsu, Bright Kellogg, Brydon Eastman, Camillo Lugaresi, Carroll Wainwright, Cary Bassin, Cary Hudson, Casey Chu, Chad Nelson, Chak Li, Chan Jun Shern, Channing Conger, Charlotte Barette, Chelsea Voss, Chen Ding, Cheng Lu, Chong Zhang, Chris Beaumont, Chris Hallacy, Chris Koch, Christian Gibson, Christina Kim, Christine Choi, Christine McLeavey, Christopher Hesse, Claudia Fischer, Clemens Winter, Coley Czarnecki, Colin Jarvis, Colin Wei, Constantin Koumouzelis, Dane Sherburn, Daniel Kappler, Daniel Levin, Daniel Levy, David Carr, David Farhi, David Mely, David Robinson, David Sasaki, Denny Jin, Dev Valladares, Dimitris Tsipras, Doug Li, Duc Phong Nguyen, Duncan Findlay, Edede Oiwoh, Edmund Wong, Ehsan Asdar, Elizabeth Proehl, Elizabeth Yang, Eric Antonow, Eric Kramer, Eric Peterson, Eric Sigler, Eric Wallace, Eugene Brevdo, Evan Mays, Farzad Khorasani, Felipe Petroski Such, Filippo Raso, Francis Zhang, Fred von Lohmann, Freddie Sulit, Gabriel Goh, Gene Oden, Geoff Salmon, Giulio Starace, Greg Brockman, Hadi Salman, Haiming Bao, Haitang Hu, Hannah Wong, Haoyu Wang, Heather Schmidt, Heather Whitney, Heewoo Jun, Hendrik Kirchner, Henrique Ponde de Oliveira Pinto, Hongyu Ren, Huiwen Chang, Hyung Won Chung, Ian Kivlichan, Ian O’Connell, Ian O’Connell, Ian Osband, Ian Silber, Ian Sohl, Ibrahim Okuyucu, Ikai Lan, Ilya Kostrikov, Ilya Sutskever, Ingmar Kanitscheider, Ishaan Gulrajani, Jacob Coxon, Jacob Menick, Jakub Pachocki, James Aung, James Betker, James Crooks, James Lennon, Jamie Kiros, Jan Leike, Jane Park, Jason Kwon, Jason Phang, Jason Teplitz, Jason Wei, Jason Wolfe, Jay

- Chen, Jeff Harris, Jenia Varavva, Jessica Gan Lee, Jessica Shieh, Ji Lin, Jiahui Yu, Jiayi Weng, Jie Tang, Jieqi Yu, Joanne Jang, Joaquin Quinonero Candela, Joe Beutler, Joe Landers, Joel Parish, Johannes Heidecke, John Schulman, Jonathan Lachman, Jonathan McKay, Jonathan Uesato, Jonathan Ward, Jong Wook Kim, Joost Huizinga, Jordan Sitkin, Jos Kraaijeveld, Josh Gross, Josh Kaplan, Josh Snyder, Joshua Achiam, Joy Jiao, Joyce Lee, Juntang Zhuang, Justyn Harriman, Kai Fricke, Kai Hayashi, Karan Singhal, Katy Shi, Kavín Karthik, Kayla Wood, Kendra Rimbach, Kenny Hsu, Kenny Nguyen, Keren Gu-Lemberg, Kevin Button, Kevin Liu, Kiel Howe, Krithika Muthukumar, Kyle Luther, Lama Ahmad, Larry Kai, Lauren Itow, Lauren Workman, Leher Pathak, Leo Chen, Li Jing, Lia Guy, Liam Fedus, Liang Zhou, Lien Mamitsuka, Lilian Weng, Lindsay McCallum, Lindsey Held, Long Ouyang, Louis Feuvrier, Lu Zhang, Lukas Kondraciuk, Lukasz Kaiser, Luke Hewitt, Luke Metz, Lyric Doshi, Mada Aflak, Maddie Simens, Madelaine Boyd, Madeleine Thompson, Marat Dukhan, Mark Chen, Mark Gray, Mark Hudnall, Marvin Zhang, Marwan Aljubei, Mateusz Litwin, Matthew Zeng, Max Johnson, Maya Shetty, Mayank Gupta, Meghan Shah, Mehmet Yatbaz, Meng Jia Yang, Mengchao Zhong, Mia Glaese, Mianna Chen, Michael Janner, Michael Lampe, Michael Petrov, Michael Wu, Michele Wang, Michelle Fradin, Michelle Pokrass, Miguel Castro, Miguel Oom Temudo de Castro, Mikhail Pavlov, Miles Brundage, Miles Wang, Minal Khan, Mira Murati, Mo Bavarian, Molly Lin, Murat Yesildal, Nacho Soto, Natalia Gimelshein, Natalie Cone, Natalie Staudacher, Natalie Summers, Natan LaFontaine, Neil Chowdhury, Nick Ryder, Nick Stathas, Nick Turley, Nik Tezak, Niko Felix, Nithanth Kudige, Nitish Keskar, Noah Deutsch, Noel Bundick, Nora Puckett, Ofir Nachum, Ola Okelola, Oleg Boiko, Oleg Murk, Oliver Jaffe, Olivia Watkins, Olivier Godement, Owen Campbell-Moore, Patrick Chao, Paul McMillan, Pavel Belov, Peng Su, Peter Bak, Peter Bakkum, Peter Deng, Peter Dolan, Peter Hoeschele, Peter Welinder, Phil Tillet, Philip Pronin, Philippe Tillet, Prafulla Dhariwal, Qiming Yuan, Rachel Dias, Rachel Lim, Rahul Arora, Rajan Troll, Randall Lin, Rapha Gontijo Lopes, Raul Puri, Reah Miyara, Reimar Leike, Renaud Gaubert, Reza Zamani, Ricky Wang, Rob Donnelly, Rob Honsby, Rocky Smith, Rohan Sahai, Rohit Ramchandani, Romain Huet, Rory Carmichael, Rowan Zellers, Roy Chen, Ruby Chen, Ruslan Nigmatullin, Ryan Cheu, Saachi Jain, Sam Altman, Sam Schoenholz, Sam Toizer, Samuel Miserendino, Sandhini Agarwal, Sara Culver, Scott Ethersmith, Scott Gray, Sean Grove, Sean Metzger, Shamez Hermani, Shantanu Jain, Shengjia Zhao, Sherwin Wu, Shino Jomoto, Shirong Wu, Shuaiqi, Xia, Sonia Phene, Spencer Papay, Srinivas Narayanan, Steve Coffey, Steve Lee, Stewart Hall, Suchir Balaji, Tal Broda, Tal Stramer, Tao Xu, Tarun Gogineni, Taya Christianson, Ted Sanders, Tejal Patwardhan, Thomas Cunningham, Thomas Degry, Thomas Dimson, Thomas Raoux, Thomas Shadwell, Tianhao Zheng, Todd Underwood, Todor Markov, Toki Sherbakov, Tom Rubin, Tom Stasi, Tomer Kaftan, Tristan Heywood, Troy Peterson, Tyce Walters, Tyna Eloundou, Valerie Qi, Veit Moeller, Vinnie Monaco, Vishal Kuo, Vlad Fomenko, Wayne Chang, Weiye Zheng, Wenda Zhou, Wesam Manassra, Will Sheu, Wojciech Zaremba, Yash Patil, Yilei Qian, Yongjik Kim, Youlong Cheng, Yu Zhang, Yuchen He, Yuchen Zhang, Yujia Jin, Yunxing Dai, and Yury Malkov. Gpt-4o system card, 2024. URL <https://arxiv.org/abs/2410.21276>.
- [43] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.
- [44] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. In *Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track*, 2024. URL <https://arxiv.org/abs/2406.17557>.
- [45] Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- [46] John Schulman. Approximating KL divergence. <http://joschu.net/blog/kl-approx.html>, 2017. Blog post. Accessed: 2026-02-09.
- [47] Thibault Sellam, Dipanjan Das, and Ankur Parikh. BLEURT: Learning robust metrics for text generation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7881–7892. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.704. URL <https://aclanthology.org/2020.acl-main.704/>.

- [48] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- [49] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*, 2024.
- [50] Lin Shi, Chiyu Ma, Wenhua Liang, Xingjian Diao, Weicheng Ma, and Soroush Vosoughi. Judging the judges: A systematic study of position bias in llm-as-a-judge. In *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, pages 292–314, 2025.
- [51] Joar Skalse, Nikolaus Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. *Advances in Neural Information Processing Systems*, 35:9460–9471, 2022.
- [52] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul Christiano. Learning to summarize with human feedback. In *Advances in Neural Information Processing Systems*, 2020.
- [53] Nisan Stiennon, Long Ouyang, Jeff Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul Christiano. Learning to summarize from human feedback, 2022. URL <https://arxiv.org/abs/2009.01325>.
- [54] Yunhao Tang and Rémi Munos. On a few pitfalls in kl divergence gradient estimation for rl, 2025. URL <https://arxiv.org/abs/2506.09477>.
- [55] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding, 2018. URL <https://arxiv.org/abs/1807.03748>.
- [56] Zengzhi Wang, Fan Zhou, Xuefeng Li, and Pengfei Liu. Octothinker: Mid-training incentivizes reinforcement learning scaling, 2025. URL <https://arxiv.org/abs/2506.20512>.
- [57] Yi Xu, Laura Ruis, Tim Rocktäschel, and Robert Kirk. Investigating non-transitivity in llm-as-a-judge. *arXiv preprint arXiv:2502.14074*, 2025.
- [58] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models, 2025. URL <https://arxiv.org/abs/2401.10020>.
- [59] Michael JQ Zhang, Zhilin Wang, Jena D Hwang, Yi Dong, Olivier Delalleau, Yejin Choi, Eunsol Choi, Xiang Ren, and Valentina Pyatkin. Diverging preferences: When do annotators disagree and do models know? *arXiv preprint arXiv:2410.14632*, 2024.
- [60] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623, 2023.
- [61] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyang Luo, Zhangchi Feng, and Yongqiang Ma. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand, 2024. Association for Computational Linguistics. URL <http://arxiv.org/abs/2403.13372>.
- [62] Fan Zhou, Zengzhi Wang, Nikhil Ranjan, Zhoujun Cheng, Liping Tang, Guowei He, Zhengzhong Liu, and Eric P. Xing. MegaMath: Pushing the limits of open math corpora, 2025. URL <https://arxiv.org/abs/2504.02807>.
- [63] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models, 2023. URL <https://arxiv.org/abs/2311.07911>.

Appendices

Table of Contents

A	Limitations and Broader Impacts	19
B	Reproducibility: Dataset and Code	19
C	LLM Usage	19
D	Extended Related Works	19
D.1	Cost Estimate of Current RLHF Pipeline	19
E	Extended Details on RM training	21
E.1	Dataset Parameters and Examples	21
E.2	Contamination Analysis	25
E.3	Proxy-Judge Analysis of Training Signal Quality	25
E.4	Sentence-aware Splitting	26
E.5	RM Training Hyperparameters	26
E.6	Continued-Pretraining Control	27
F	Curated Dataset Details for Token-Matched Comparison	27
F.1	Dataset Construction and Token-Equivalent Pair Counts	27
G	Extended Details on BoN	28
G.1	BoN Generation and Evaluation Hyperparameters	28
H	Extended Results for RM Training	29
H.1	Validation Reward Curve	29
H.2	Multi-Seed Robustness	30
H.3	RewardBench and RewardBench 2 Per-Subset Gains	31
H.4	Safety Transfer Beyond Refusal Wording	31
H.5	Transfer to Tool-Use Preference Evaluation	32
H.6	Size-matched Reward Model Ranking	32
I	GRPO Objective and Actor-Training Details	35
I.1	GRPO Objective	35
I.2	Training Setup and Results	35
J	Extended Results for Continuing Training on Curated Preference Dataset	36
J.1	Label Efficiency of NTP Initialization	37
K	Licenses for Existing Assets	37
L	Compute Resources	38

A Limitations and Broader Impacts

Although NTP is competitive with supervised reward training under a matched token budget, our models still underperform the strongest reward models trained on large curated preference datasets in many settings (Table 1 and Section 3.4). We therefore do not claim that NTP replaces supervised reward training. However, our mid-training experiments show that NTP can improve the label efficiency of supervised training by providing an effective initialization learned from unlabeled web text. With only 50% of the labeled RLHFMix1 data, NTP initialization surpasses base initialization trained on the full dataset and matches its own full-data performance on Reward-Bench 2 (Section 3.6 and Appendix J.1). This result supports using unsupervised reward learning to substantially reduce the amount of annotated data needed for subsequent training while retaining the benefits of high-quality preference supervision.

The broader impact of this work is to make reward-model training more accessible and scalable by reducing the cost of constructing preference data (Appendix D.1). NTP constructs comparisons directly from existing web text without candidate generation or human or model-generated preference judgments. Together with the improved label efficiency of subsequent supervised training, this offers a way for limited annotation budgets to support stronger reward models. However, continuation consistency is an imperfect proxy for response quality: web text can contain factual errors, harmful content, and social biases that the learned reward function may reproduce. Better benchmark performance does not guarantee safe behavior under downstream optimization. Application-specific evaluation, checks for biased preferences and reward exploitation, and targeted human feedback remain important when deploying these models.

B Reproducibility: Dataset and Code

The study uses publicly available datasets listed in Appendix K. Our code is available at <https://github.com/jingxuanf0214/rm-scaling>. The algorithm, evaluation protocols, and training details are documented in Section 2 and Appendices E, G, I and J.

C LLM Usage

LLMs were used in a supporting role during the preparation of this paper. Specifically, we used LLM assistance for: (1) checking grammar errors and editing text for clarity and readability; and (2) helping write code to visualize experimental results (e.g., plotting scripts for figures). LLM assistance was not used for any part of the core research methodology and algorithm design.

D Extended Related Works

D.1 Cost Estimate of Current RLHF Pipeline

RLHF relies critically on the availability of high-quality preference datasets, but collecting those datasets is widely viewed as one of the dominant bottlenecks in the RLHF workflow—spanning prompt curation, candidate generation, preference labeling, and iterative refinement. This is emphasized in [16], which lays out end-to-end RLHF pipelines and highlights how feedback collection/labeling becomes a central practical constraint when scaling RLHF beyond small offline datasets.

In this context, we explicitly quantify the cost of the full preference-data workflow focusing primarily on candidate generation (a prompt is used to generate a candidate pool) and preference anno-

tation (a judge assigns which candidate is chosen/rejected) using representative datasets used by the [16] RLHF workflow.

Because the underlying datasets may use a mixture of completion models and judge models, we report costs by instantiating token volumes under a single representative model class for each stage. Specifically, for both candidate generation and LLM judging, we assume a GPT-4-class model for demonstration (e.g., gpt-4o [42]), not because it necessarily reflects the exact model used in data collection, but because it provides a consistent reference point for pricing across datasets. This assumption should be viewed as an average-cost proxy—rather than the most advanced or specialized reasoning model—while still being strong enough to produce high-quality candidate completions and reliable preference judgments.

To estimate candidate generation cost, we treat each preference pair as one prompt that yields two generated completions (preferred and rejected). Thus, each pair corresponds to a single generation call with prompt-side input (plus optional fixed overhead) and two completion outputs. Using the dataset length statistics, the generated output tokens per pair are computed as:

$$T_{\text{cand}} = \text{PrefLen} + \text{RejLen}. \quad (4)$$

We then include an assumed constant input overhead $O_{\text{in}}^{\text{gen}}$ (e.g., system message + formatting constraints). For a dataset with N preference pairs, the total token usage for candidate generation is:

$$T_{\text{in}}^{\text{gen}} = N(\text{PromptLen} + O_{\text{in}}^{\text{gen}}), \quad T_{\text{out}}^{\text{gen}} = N \cdot T_{\text{cand}}. \quad (5)$$

Finally, given per-million token prices $P_{\text{in}}^{\text{gen}}$ and $P_{\text{out}}^{\text{gen}}$ for a GPT-4-class completion model, the total candidate generation cost is:

$$\text{Cost}^{\text{gen}} = (T_{\text{in}}^{\text{gen}}/10^6)P_{\text{in}}^{\text{gen}} + (T_{\text{out}}^{\text{gen}}/10^6)P_{\text{out}}^{\text{gen}}. \quad (6)$$

In Table 7, we report candidate generation cost estimates based on the above calculations Eq. (6) for the representative preference dataset included in [16]. Costs are computed per dataset using the publicly listed gpt-4o text rates $P_{\text{in}} = \$2.50$ and $P_{\text{out}} = \$10.00$ per 1M tokens. We additionally report the aggregate total obtained by summing costs across datasets, reflecting the common RLHF practice of training on the union of these curated preference corpora.

Table 7: Estimated candidate generation cost instantiated with a GPT-4-class completion model gpt-4o.

Dataset	#Pairs N	Prompt length	Preferred length	Rejected length	T_{cand}	Overhead (in/out)	$T_{\text{in}}^{\text{gen}}$	$T_{\text{out}}^{\text{gen}}$	Cost (\$)	\$/pair
HH-RLHF	115,396	160.4	82.2	73.6	155.8	0 / -	18.51M	17.98M	180.85	0.00157
SHP	93,301	186.2	173.6	88.8	262.4	0 / -	17.37M	24.48M	230.60	0.00247
HelpSteer	37,131	530.0	116.4	89.3	205.7	0 / -	19.68M	7.64M	100.46	0.00271
PKU-SafeRLHF-30K	26,874	21.5	70.4	74.6	145.0	0 / -	0.58M	3.90M	32.33	0.00120
UltraFeedback	340,025	161.5	279.5	211.1	490.6	0 / -	54.91M	166.82M	1444.36	0.00425
UltraInteract	161,927	507.4	396.6	416.7	813.3	0 / -	82.16M	131.70M	1217.89	0.00752
CodeUltraFeedback	50,156	172.8	427.6	400.6	828.2	0 / -	8.67M	41.54M	349.65	0.00697
Argilla-Math	2,418	36.5	276.5	265.3	541.8	0 / -	0.09M	1.31M	10.66	0.00441
OpenOrca	6,926	153.3	165.4	260.5	425.9	0 / -	1.06M	2.95M	25.72	0.00371
Capybara	14,811	634.5	348.4	401.9	750.3	0 / -	9.40M	11.11M	107.70	0.00727
[16]	-	-	-	-	-	-	-	-	3700.22	-

To estimate the LLM annotation cost (assuming single judge), we treat each preference pair as one judge call whose input is the concatenation of the prompt and the two candidate responses plus

a fixed rubric/formatting overhead, and whose output is a short structured decision. The content tokens per pair are computed as:

$$T_{\text{content}} = \text{PromptLen} + \text{PrefLen} + \text{RejLen}. \quad (7)$$

We then include an assumed constant input overhead $O_{\text{in}}^{\text{judge}}$ tokens and output budget $O_{\text{out}}^{\text{judge}}$ tokens (e.g., JSON verdict + brief rationale). For a dataset with N labeled pairs, total token usage is:

$$T_{\text{in}}^{\text{judge}} = N(T_{\text{content}} + O_{\text{in}}^{\text{judge}}), \quad T_{\text{out}}^{\text{judge}} = N \cdot O_{\text{out}}^{\text{judge}}. \quad (8)$$

Given per-million token prices $P_{\text{in}}^{\text{judge}}, P_{\text{out}}^{\text{judge}}$ for a GPT-4-class judge, the total cost is:

$$\text{Cost} = (T_{\text{in}}^{\text{judge}}/10^6)P_{\text{in}}^{\text{judge}} + (T_{\text{out}}^{\text{judge}}/10^6)P_{\text{out}}^{\text{judge}}. \quad (9)$$

Similar to Table 7, Table 8 reports candidate annotation cost estimates based on calculation of Eq. (9) for the representative preference dataset included in [16].

Table 8: Estimated LLM annotation cost for preference labeling with a single GPT-4-class judge (gpt-4o pricing). We use $O_{\text{in}} = 40$ input overhead tokens and $O_{\text{out}} = 40$ output tokens per pair.

Dataset	#Pairs N	Prompt length	Preferred length	Rejected length	T_{content}	Overhead (in/out)	$T_{\text{in}}^{\text{judge}}$	$T_{\text{out}}^{\text{judge}}$	Cost (\$)	\$/pair
HH-RLHF	115,396	160.4	82.2	73.6	316.2	40 / 40	41.10M	4.62M	148.92	0.00129
SHP	93,301	186.2	173.6	88.8	448.6	40 / 40	45.59M	3.73M	151.29	0.00162
HelpSteer	37,131	530.0	116.4	89.3	735.7	40 / 40	28.79M	1.49M	86.91	0.00234
PKU-SafeRLHF-30K	26,874	21.5	70.4	74.6	166.5	40 / 40	5.55M	1.07M	24.57	0.00091
UltraFeedback	340,025	161.5	279.5	211.1	652.1	40 / 40	235.71M	13.60M	725.83	0.00213
UltraInteract	161,927	507.4	396.6	416.7	1320.7	40 / 40	220.56M	6.48M	616.28	0.00381
CodeUltraFeedback	50,156	172.8	427.6	400.6	1001.0	40 / 40	52.20M	2.01M	150.52	0.00300
Argilla-Math	2,418	36.5	276.5	265.3	578.3	40 / 40	1.50M	0.10M	4.74	0.00196
OpenOrca	6,926	153.3	165.4	260.5	579.2	40 / 40	4.29M	0.28M	13.53	0.00195
Capybara	14,811	634.5	348.4	401.9	1384.8	40 / 40	21.51M	0.59M	56.53	0.00382
[16]	–	–	–	–	–	–	–	–	1979.10	–

E Extended Details on RM training

E.1 Dataset Parameters and Examples

For *FineMath*, we use the *FineMath-4plus* subset released by HuggingFace [2]. *4plus* indicates that they are the selective higher quality entries in the dataset (as opposed to *3plus*). A representative data example is shown in 8.

For the general-domain experiment, we use *FineWeb-Edu* [44], the educational subset of FineWeb selected by an educational-quality classifier. An example excerpt from the public release is shown in Fig. 9.

For the additional mathematical corpus, we use *MegaMath-Web-Pro-Max* from OctoThinker [56]. This release filters MegaMath-Web using a fastText classifier trained on Llama-3.1-70B-Instruct quality annotations, then refines the selected documents with the same LLM. We construct our in-batch comparisons from these released texts without generating preference labels. An example entry from the public release is shown in Fig. 10.

We use fixed-length chunks of $L = 1536$ tokens, with a default 1:2 split into a prompt prefix of $L_1 = 512$ tokens and a continuation of $L_2 = 1024$ tokens, in the notation of [Section 2](#). These lengths are fixed across all main experiments. The split is determined by token position and need not coincide with a sentence boundary.

To assess sensitivity to split position, we keep the total length fixed at $L = 1536$ and compare a shorter prefix (1:2), equal-length segments (1:1), and a longer prefix (2:1), corresponding to $(L_1, L_2) = (512, 1024)$, $(768, 768)$, and $(1024, 512)$, respectively. All three configurations yield positive RewardBench 2 gains, averaging $+7.87 \pm 0.96$, with no clear trend relating the split ratio to performance. We therefore retain the 1:2 split as a simple default rather than treating the ratio as a critical tunable hyperparameter.

Period Of Function

by -2 views

Period of sinusoidal functions from equation. You can figure this out without looking at a graph by dividing with the frequency which in this case is 2.

Periodic functions are used throughout science to describe oscillations, waves and other phenomena that exhibit periodicity.

This is why this function family is also called the periodic function family. Therefore in the case of the basic cosine function $f(x)$, when this occurs we call the horizontal shift the period of the function. If a function has a repeating pattern like sine or cosine it is called a periodic function. Amplitude a : let b be a real number. Horizontal stretch is measured for sinusoidal functions as their periods.

For basic sine and cosine functions the period is 2π . Any function that is not periodic is called aperiodic. The period is defined as the length of one wave of the function. The period is the length on the x -axis in one cycle. The period of a sinusoid is the length of a complete cycle.

More formally, we say that this type of function has a positive constant k where any input x :

$$f(x + k) = f(x)$$

The period is defined as the length of a function's cycle. According to the periodic function definition, the period of a function is represented as $f(x) = f(x + p)$, where p is the real number equal to the period of the given function $f(x)$.

Where $f(x + P) = f(x)$ for all values of x . The period of a periodic function is the interval of x -values on which the cycle of the graph that is repeated in both directions lies. In this case, one full wave is 180 degrees or π radians. So the period of $\sin$ or $\cos$ is 2π .

The period of the function can be calculated using:

$$P = \frac{2\pi}{|b|}$$

In the problems below we will use the formula for the period P of trigonometric functions of the form $y = a \sin(bx + c) + d$ or $y = a \cos(bx + c) + d$.

The amplitude of $y = a \sin x$ and $y = a \cos x$ represents half the distance between the maximum and minimum values of the function. The period is the length of the smallest interval that contains exactly one copy of the repeating pattern. A periodic function repeats its values at set intervals called periods.

Figure 8: *FineMath-4plus* dataset example.

Hold the salt: UCLA engineers develop revolutionary new desalination membrane

Process uses atmospheric pressure plasma to create filtering 'brush layer'

Desalination can become more economical and used as a viable alternate water resource.

By Wileen Wong Kromhout

Originally published in UCLA Newsroom

Researchers from the UCLA Henry Samueli School of Engineering and Applied Science have unveiled a new class of reverse-osmosis membranes for desalination that resist the clogging which typically occurs when seawater, brackish water and waste water are purified.

The highly permeable, surface-structured membrane can easily be incorporated into today's commercial production system, the researchers say, and could help to significantly reduce desalination operating costs. Their findings appear in the current issue of the Journal of Materials Chemistry.

Reverse-osmosis (RO) desalination uses high pressure to force polluted water through the pores of a membrane. While water molecules pass through the pores, mineral salt ions, bacteria and other impurities cannot. Over time, these particles build up on the membrane's surface, leading to clogging and membrane damage. This scaling and fouling places higher energy demands on the pumping system and necessitates costly cleanup and membrane replacement.

...

Figure 9: *FineWeb-Edu* dataset example (excerpt from [UCLA Newsroom](#)).

Scalars and Vectors

Mathematics is the language of physics, allowing us to quantitatively describe the world. In mechanics, we use two types of quantities: scalars and vectors.

Goals:

- Learn the definitions of scalars and vectors
- See examples of vector and scalar quantities
- Translate vectors to and from their components
- Understand how to add, subtract, and multiply vectors

Definitions:

- Scalar: A scalar describes a one-dimensional quantity, requiring only one number to completely describe it. Examples include temperature, time, speed, mass, and location along a line (1D).
- Vector: A vector describes a multi-dimensional quantity, requiring more than one number to completely describe it. Vectors have two characteristics: magnitude and direction. Examples include location in a plane (2D), location in space (3D), velocity, acceleration, and force.

Vector Diagrams:

Vector diagrams visualize physical systems. Although problems can be solved algebraically, diagrams help identify subtleties. The first step in solving physics problems is to draw a picture. Vector diagrams show the concept of the tip and tail of a vector, with direction indicated by an arrowhead. Two vectors with the same magnitude and direction are considered the same, allowing them to be moved around the coordinate system to simplify math.

Expressing Vectors Using Components:

Vectors can be expressed in terms of magnitude and direction or components. The magnitude is the vector's length, and direction is given in terms of an angle. Vectors are often broken down into component vectors, which run parallel to the coordinate axes. A two-dimensional vector has two component vectors: one in the X direction and one in the Y direction.

Vector Addition:

Vector addition involves moving the tail of the second vector to the tip of the first vector to get the resultant vector. Vectors can be added by expressing them as independent components and adding corresponding components.

Multiplication of a Vector by a Scalar:

A vector can be multiplied by a scalar by multiplying each component by that number. The vector's direction remains the same, but its length changes. If $A = (1, 2)$, then $3A = (3, 6)$. Multiplying a vector by -1 reverses its direction.

Vector Subtraction:

Vector subtraction works like vector addition, except the vector being subtracted is multiplied by -1 . This is similar to subtracting numbers: $A - B = A + (-B)$. The order of vectors is important, as reversing them results in a vector of opposite direction.

Example 1:

A hiker walks 53.1 degrees north of east for 2.5 km and then due east for 2.0 km. What is her total displacement from the starting point, measured along a straight line?

Example 2:

A telephone pole support cable must be moved 2 meters closer to the pole. If the pole is 10 meters tall and the cable is currently fastened to the ground 8 meters from the pole, how much will the workers need to cut off from the cable when they move it?

Figure 10: *MegaMath-Web-Pro-Max* dataset example from the OctoThinker release. The entry contains LLM-refined text; its recorded source is an [IUPUI tutorial on scalars and vectors](#).

E.2 Contamination Analysis

FineMath-4plus, used for our main experiments, was released as part of the SmolLM2 data pipeline [2], which reports 13-gram decontamination against several standard math and reasoning benchmarks, including GSM8K, MATH, MMLU, ARC-E, and ARC-C. Since GSM8K and MATH/-MATH500 are among our main downstream math evaluations, this provides an existing contamination control for the core math benchmarks used in our study.

By contrast, *FineWeb-Edu* [44] inherits FineWeb’s within-crawl MinHash deduplication based on 5-grams, but its release does not report benchmark-specific decontamination.

MegaMath-Web-Pro-Max [56] is obtained by filtering and refining MegaMath-Web. The original MegaMath release [62] reports removing documents with exact 13-gram overlap against 12 math benchmarks, including GSM8K, MATH, MMLU, and AIME.

We further perform our own 13-gram contamination audit between the exact training subsets drawn from all three corpora and their corresponding evaluation benchmarks. For *FineMath-4plus*, we audit RewardBench, IFEval, and ToxiGen, which are not covered by its source release; for *FineWeb-Edu* and *MegaMath-Web-Pro-Max*, we audit RewardBench 2. We find no 13-gram overlap in any of these corpus–benchmark comparisons. These results suggest that the reported gains are unlikely to be explained by direct benchmark leakage under this standard n -gram overlap criterion. We will release the contamination-checking logs together with the code and training data processing scripts.

E.3 Proxy-Judge Analysis of Training Signal Quality

Section 2 argues that in-batch cross-pairs serve as reasonable hard negatives on average. These comparisons target contextual compatibility: another continuation can occasionally fit the prefix, and the observed continuation is not guaranteed to be correct. However, the difficulty of a batch’s implicit preference signal is not uniform: it depends on how contextually similar the in-batch negatives happen to be to each specific prefix. When negatives are drawn from passages that are topically or stylistically distant from the target prefix, the true continuation may be trivially distinguishable, providing little useful ranking supervision. Conversely, when negatives come from passages sharing close context—as encouraged by our non-sentence-aware splitting and larger batch sizes—the comparison becomes genuinely hard. This batch-level variation in signal difficulty can contribute to the noisy or non-monotonic gains observed during training, and provides an additional motivation for the design choices we ablate in Section 3.2: larger batches increase the proportion of contextually similar in-batch negatives quadratically, and non-sentence-aware splitting retains more contiguous chunks from the same underlying passage, both of which raise the average difficulty of the training signal.

To quantify this effect, we conduct a diagnostic experiment using an external LLM as a proxy judge. We sample 100 training batches from the NTP training corpus, each with batch size 32. For each example in a batch, the prefix is paired with its true suffix as the positive continuation and with mismatched suffixes from other examples in the same batch as negative continuations. We then ask a Llama-3.2-3B-Instruct model to identify which candidate suffix is the coherent continuation of the given prefix. The proxy-judge accuracy for a batch is computed as the fraction of examples for which the model selects the true suffix over the in-batch negatives.

The resulting batch-level accuracies vary substantially, ranging from approximately 0.3 to 0.8 across the 100 sampled batches. This indicates that the implicit preference signal is heterogeneous: in some batches the true continuation is relatively easy to distinguish from mismatched suffixes, while

in others the comparison is much more challenging. Critically, this heterogeneity also confirms that in-batch negatives are not uniformly trivial—many batches contain difficult comparisons where a small instruction-tuned model cannot reliably identify the true continuation, supporting our claim that the cross-pair construction provides a non-degenerate learning signal. Together, these results suggest that strategies which increase the proportion of hard batches—larger batch sizes and non-sentence-aware splitting—are directly beneficial for training signal quality, consistent with the ablation results in [Section 3.2](#).

E.4 Sentence-aware Splitting

The *preserve sentence* variant in [Section 3.2](#) constructs prefix-suffix pairs while respecting sentence boundaries and sequence-length constraints. Concretely, text documents are converted into training pairs by (i) normalizing and segmenting each document into sentence-like units and appending an end-of-sequence marker, (ii) tokenizing each unit, (iii) greedily packing consecutive units into token blocks up to a fixed maximum length, never splitting a unit except when it exceeds the maximum (then either split or discard), (iv) discarding blocks below a minimum effective length threshold, and (v) selecting within each retained block a unit boundary whose cumulative token count is closest to a desired prefix length to form a prefix (preceding tokens) and suffix (remaining tokens).

Enforcing sentence boundaries together with these length constraints has a side effect: packing and filtering discard many intermediate spans, leaving gaps between retained training examples in the original text. Consequently, examples that are adjacent in a batch can come from more distant passages and share less local context. Their cross-paired suffixes are then less related to the target prefix and easier to reject using coarse topic or context differences, weakening the pressure to learn fine-grained continuation consistency. Allowing sentence breaks retains contiguous fixed-length chunks from the text stream, preserving more of the local context shared by in-batch examples and yielding more confusable negatives. This change in batch composition helps explain the stronger gains of the *break sentence* variant ([Fig. 4](#)).

E.5 RM Training Hyperparameters

RM models of different backbones reported in [Section 3.1](#) are trained using open source framework `verl` [49] with our proposed [Algorithm 1](#) and hyperparameters reported in [Table 9](#). We perform a learning rate sweep across candidate sets $3e-7$, $5e-7$, $1e-6$, $3e-6$, and $1e-5$ and identify $1e-6$ as the best configuration that is consistent across initialization and backbone types.

For `Qwen2.5-7B-Instruct` we use batch size 8 which is different from the default value reported in [Section 3.2](#). This is because both instruction-tuned backbones have higher initialized performance compared to base models. In this case, scaling in-batch comparison examples provides minimal gain. Also, increasing data loader batch size under our algorithm produces quadratic increase in effective batch size, requiring more compute. Therefore, for these cases we use the smallest configuration that produces a similar performance gain.

Table 9: RM training hyperparameters.

Backbone	Seed	Optimizer	Learning rate	Batch size	c (center)
Llama-3.2-1B	2025	Adam betas 0.9, 0.95	10^{-6} , constant 0.05 warmup ratio	32	0.01
Llama-3.2-3B	2025	Adam betas 0.9, 0.95	10^{-6} , constant 0.05 warmup ratio	32	0.01
Qwen2.5-3B-Inst.	2028	Adam betas 0.9, 0.95	10^{-6} , constant 0.05 warmup ratio	32	0.01
Qwen2.5-7B-Inst.	2025	Adam betas 0.9, 0.95	10^{-6} , constant 0.05 warmup ratio	8	0.01

E.6 Continued-Pretraining Control

We perform continued pretraining (CPT) of Llama-3.2-3B on the same 11M *FineMath-4plus* tokens in the same order as NTP training, using the LLaMA-Factory framework [61] with the hyperparameters detailed in Table 10. The two readouts in Section 3.2 use 512 auxiliary perturbed preference pairs from *FineMath* text blocks excluded from the earlier reward-model and continued-pretraining runs, and are evaluated on RewardBench 2. The frozen-backbone probe uses the final-layer hidden state at the last response token, the position from which our value head reads. It fits a logistic Bradley-Terry head on the difference between the chosen and rejected states, without an intercept and with the L2 penalty chosen on a 512-pair validation split. We repeat the fit on three disjoint 512-pair subsets and report the mean. Conversion jointly finetunes a newly initialized scalar reward head and all backbone parameters on 512 auxiliary pairs with the objective of Eq. (1), 32 pairs per step at a constant learning rate of 10^{-5} for three epochs. We report the mean over three runs with independent head initializations. Applied to our 11M-token NTP reward model with its original head discarded, the probe achieves 36.4 on RewardBench 2, compared with 36.2 for the original RM head (Table 3).

Table 10: Continued-pretraining recipe for the 11M-token control in Table 3. Blocks per step is the global batch size.

Backbone	Corpus	Optimizer	Learning rate	Blocks per step	Cutoff (tokens)	Epochs
Llama-3.2-3B	FineMath-4plus	AdamW	10^{-5} peak, cosine, warmup ratio 0.1	1024	2048	1

F Curated Dataset Details for Token-Matched Comparison

F.1 Dataset Construction and Token-Equivalent Pair Counts

Section 3.1 compares our method against training on curated preference datasets under a matched 11M-token budget. Here we describe how the two datasets are constructed and annotated, followed by their token statistics and the equivalent number of preference pairs under this budget.

Skywork-Reward-Preference-80K-v0.2. We use Skywork/Skywork-Reward-Preference-80K-v0.2 [35], which contains 77,016 preference pairs over chat responses. The dataset consists primarily of Magpie-series synthetic responses ranked by ArmoRM and is decontaminated against RewardBench. Thus, it provides curated prompt-response pairs with preference labels supplied primarily by a reward model.

Based on dataset statistics, mean per-entry lengths are approximately: instruction ≈ 460 characters (≈ 115 tokens), chosen response $\approx 2,057$ characters (≈ 514 tokens), rejected response $\approx 2,337$ characters (≈ 584 tokens), yielding a mean of approximately 1,213 tokens per preference pair (using a standard ~ 4 characters per token approximation). Under our 11M-token budget, this corresponds to training on approximately $11,000,000/1,213 \approx 9,070$ pairs ($\approx 9\text{K}$ pairs).

RLHFMix1. RLHFMix1 [16] is a curated mixture of 244,216 preference pairs. The mixture uniformly samples four sources to form a balanced training set, namely UltraFeedback [13], HH-RLHF [3], Stanford Human Preferences (SHP) [21], and Summarize-from-Feedback [53]. UltraFeedback supplies GPT-4-judged preferences, while the other three sources provide human preferences. Thus, this baseline relies on both curated prompt-response pairs and external preference annotations.

Based on dataset statistics, mean per-entry lengths are approximately: instruction ≈ 896 characters (≈ 224 tokens), chosen response ≈ 594 characters (≈ 149 tokens), rejected response ≈ 481 characters (≈ 120 tokens), yielding a mean of approximately 493 tokens per preference pair. Under our 11M-token budget, this corresponds to training on approximately $11,000,000/493 \approx 22,310$ pairs ($\approx 22\text{K}$ pairs).

These token-per-pair estimates indicate that our 11M-token budget is equivalent to training on roughly 9K pairs from Skywork-80K and 22K pairs from RLHFMix1—both a small fraction of each dataset’s full size—providing a strictly controlled comparison of data-efficiency across supervision regimes.

Table 11: Per-entry token statistics for curated comparison datasets and the equivalent number of preference pairs under an 11M-token training budget. Token counts are estimated at ~ 4 characters per token.

Dataset	#Pairs	Instruction (tokens)	Chosen (tokens)	Rejected (tokens)	Tokens/pair	Equiv. pairs @ 11M tok.
Skywork-80K-v0.2	77,016	≈ 115	≈ 514	≈ 584	$\approx 1,213$	$\approx 9,070$
RLHFMix1	244,216	≈ 224	≈ 149	≈ 120	≈ 493	$\approx 22,310$

G Extended Details on BoN

G.1 BoN Generation and Evaluation Hyperparameters

Response generation for BoN is adapted from the open-source framework `lm-evaluation-harness` [24]. We use the datasets, prompts, actor generation settings, and evaluation metrics specified in Table 12. For HumanEval+, we use version 0.1.10 and retain the 160 of 164 problems whose canonical solutions pass under EvalPlus. A selected program is correct only if it passes both the base and the additional test suites [8, 37]. Alongside the two Skywork Reward V2 baselines, we evaluate PKU’s `beaver-7b-unified-reward` [14, 31] on the same actor-generated candidate sets.

Table 12: BoN response generation and evaluation details by task.

Task (ID/OOD)	Dataset / split	Prompt format	Actor decoding	Accuracy / scoring criterion
MATH500 (ID)	HuggingFaceH4/ MATH-500 (test)	Problem: question. Provide your reasoning step by step, then the final answer the form FINAL_ANSWER: answer .	$T=1.0$, top- $p=1.0$ max tokens=512	lm-eval-harness math-verify
GSM8K (ID)	openai/gsm8k (test)	Problem: question. Provide your reasoning step by step, then the final answer the form FINAL_ANSWER: answer .	$T=1.0$, top- $p=1.0$ max tokens=512	lm-eval-harness math-verify
HumanEval+ (OOD)	evalplus/ humanevalplus v0.1.10 (160 of 164 problems whose canonical solutions pass under EvalPlus)	Please provide a self-contained Python script that solves the following problem in a markdown code block: problem	$T=1.0$, top- $p=1.0$ max tokens=512	evalplus; pass iff the extracted program passes both the base and the plus test suites
ToxiGen (OOD)	toxigen/toxigen-data (test)	Is the following statement hateful? Statement: text. Respond with either Yes or No. Response:	$T=1.0$, top- $p=1.0$ max tokens=20	lm-eval-harness
IFEval (OOD)	google/IFEval (train)	prompt	$T=1.0$, top- $p=1.0$ max tok.=1280	lm-eval-harness prompt level strict

H Extended Results for RM Training

H.1 Validation Reward Curve

In Section 2 and Section 3.2, we motivate the addition of centering loss by claiming that it constrains reward magnitudes and reduces uncontrolled scale drift. Figure 11 shows the raw positive and total reward scores on the validation set for with and without centering training configs. For both cases, raw reward difference shows an increasing trend and gets stabilized around 1.5-2 million tokens but the with centering loss case constrains the reward difference to much smaller range and maintains total reward close to zero. To provide some intuition of this centering regularization for our RM training case: we want the model to be able to learn good continuation signals from the raw text, i.e. reliably separating chosen and rejected samples, but we in general don't want to give scores of large magnitude to inherently noisy data. In fact, when running a high-quality of-the-shelf RM Skywork-Reward-V2-Llama-3.1-8B on an example subset of our data, we observe that it is able to score chosen and rejected samples on average differently but with a difference margin of also around 4 which resembles the margin we see in the centering loss training case.

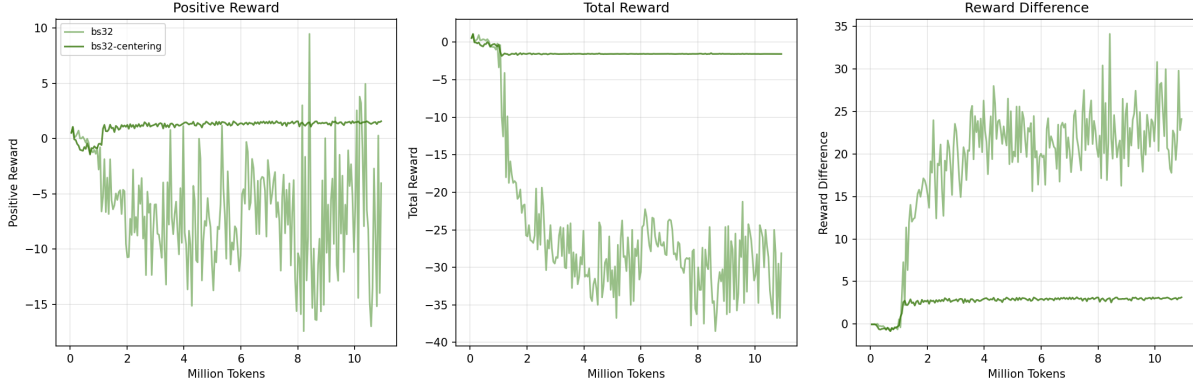


Figure 11: Validation reward curve with or without centering loss.

H.2 Multi-Seed Robustness

Random initialization affects the starting accuracy of an RM, consistent with previous reports [36]. Evaluating five initialization seeds for Llama-3.2-3B on RewardBench 2 gives a mean initial accuracy of approximately 0.26, with a 95% confidence interval of [0.223, 0.297]; the initialization used in our main experiments falls within this interval. This variability motivates measuring improvement relative to each run’s own initialization. Accordingly, our RewardBench and RewardBench 2, BoN, and actor-training evaluations include initialized RM baselines to distinguish the contribution of training from the starting performance.

To assess whether the gains from training are robust to seed choice, we run the default setup (Llama-3.2-3B, *FineMath-4plus*, 11M-token budget) under three independently sampled seeds and report RewardBench 2 gains over each run’s corresponding initialization in Table 13. All three runs yield positive overall gains within a narrow range of +7.2 to +8.4 percentage points, with a mean of $+7.7 \pm 0.35$ (standard error). The main-paper seed (2025) yields a representative gain of +7.7 points. Thus, despite variation in starting accuracy, the consistent improvements over the corresponding initializations support a reproducible learning effect from NTP training. Both overall and in-domain gains are consistent across seeds, with ID gains ranging from +16.1 to +19.1 points. Greater variability appears in how OOD gains are distributed between safety and the remaining subsets.

Table 13: RewardBench 2 gains in percentage points over each run’s initialization for three independent runs of the default Llama-3.2-3B setup. The final row reports means across seeds; the overall gain is reported as mean $\pm$ standard error.

Seed	ID	OOD Safety	OOD Others	RewardBench 2 Avg
2025	+16.1	+5.4	+7.4	+7.7
2027	+17.5	+10.5	+4.7	+8.4
2028	+19.1	+16.1	+0.2	+7.2
Mean	+17.6	+10.7	+4.1	$+7.7 \pm 0.35$

H.3 RewardBench and RewardBench 2 Per-Subset Gains

To clarify which preference dimensions improve beyond the grouped results in Fig. 2, Table 14 reports per-subset gains for the default Llama-3.2-3B setup trained on 11M tokens of *FineMath-4plus*. Each value is the mean improvement over the corresponding initialization across three seeds detailed in Table 13.

Table 14: RewardBench and RewardBench 2 subset gains in percentage points over each run’s initialization, averaged across three seeds for NTP training of Llama-3.2-3B on 11M FineMath tokens.

Benchmark	Subset	Gain
RewardBench	Chat	+4.9
	Chat Hard	+7.0
	Safety	+16.0
	Reasoning	+13.1
RewardBench 2	Math	+17.6
	Safety	+10.7
	Factuality	+2.9
	Focus	+10.1
	Precise IF	+2.2

On RewardBench, the largest gain is on Safety (+16.0), exceeding the gain on the in-domain Reasoning subset (+13.1). Improvements on Chat (+4.9) and Chat Hard (+7.0) further indicate transfer to response helpfulness and conversational preferences. On RewardBench 2, Math improves most (+17.6), but Safety (+10.7) and Focus (+10.1) also show substantial gains. These results indicate that NTP training on math-related text improves judgments of appropriate responses to harmful and benign but potentially risky requests, as well as relevance to the user’s request, despite receiving no explicit preference labels for these capabilities.

Transfer is uneven, with on average smaller gains on Factuality (+2.9) and Precise IF (+2.2). Thus, the subset breakdown supports a reward signal that transfers across several response-quality dimensions, with particularly strong improvements in safety and relevance beyond mathematical reasoning; it does not imply equally strong gains on every preference dimension.

H.4 Safety Transfer Beyond Refusal Wording

The safety gains in Section 3.1 and Appendix H.3 raise the question of whether NTP improves judgments of response content or mainly rewards superficial refusal cues. We examine this distinction using a category-level breakdown and a diagnostic subset of RewardBench 2 [40].

Category-level breakdown. Using the CoCoNot-style taxonomy of RewardBench 2, approximately 85% of the observed Safety gain comes from privacy violations, misinformation, false presuppositions, and copyright violations. These categories concern the content and implications of a response, but their contribution alone does not rule out reliance on stylistic cues. We therefore also examine examples in which refusal wording is misleading.

Misleading refusal wording. For the seed-2027 Llama-3.2-3B run trained on 11M *FineMath-4plus* tokens, the overall Safety gain is 10.5 percentage points over initialization (Table 13). Within this evaluation, we identify 43 “fake-refusal” examples whose rejected response begins with refusal boilerplate but ultimately complies with the harmful request. The model improves by 8.9 points on these examples, compared with 10.7 points on the remaining 407 examples, with both gains

measured against the same run’s initialization. Thus, the safety improvement persists when the presence of refusal language alone would favor the rejected response.

These results provide evidence against a purely stylistic explanation of safety transfer. One possible explanation is that continuation discrimination makes semantic distinctions in prompt–response compatibility more accessible to the reward function. The analysis does not establish this mechanism or rule out a contribution from style: the fake-refusal result covers a small subset from one seed and should be interpreted as diagnostic evidence.

H.5 Transfer to Tool-Use Preference Evaluation

To probe whether the learned reward signal transfers to judging tool use, we evaluate `FineMath-RM-Llama-3.2-3B` on `FC-RewardBench` [1]. The benchmark contains 1,500 examples derived from the single-turn splits of `BFCL-v3`, each providing a user query, a catalog of available tools, and correct and incorrect function-call outputs. Pairwise accuracy measures how often the reward model assigns a higher score to the correct output.

Our NTP-trained 3B reward model achieves 68.7% accuracy, compared with 75.9% for the size-matched `Skywork-Reward-V2-Llama-3.2-3B`. This provides evidence of transfer to tool-use preference judgment despite training on mathematical web text without explicit tool-use preference supervision. Both models remain substantially below the specialized `ToolRM-7B` and `ToolRM-14B` models, which achieve approximately 90% in Agarwal et al. [1]. The remaining gap highlights the benefit of training on tool-use preference data, although those specialized models also have larger backbones.

This evaluation concerns preference judgments over function-call outputs from single-turn inputs. It does not establish the ability to evaluate interactive, long-horizon agent trajectories. Extending NTP to trajectory-structured corpora and evaluating rewards for multi-step agent behavior remain directions for future work.

H.6 Size-matched Reward Model Ranking

In Section 3.1, we report trained RMs’ `RewardBench` and `RewardBench 2` size-matched rankings. Here in Tables 15 to 20 we provide the exact size-matched rankings with our trained RMs inserted.

Table 15: `RewardBench` leaderboard RMs in the 0.5–1.5B parameter range, sorted by AvgAcc.

Model	Params (B)	AvgAcc	Type
<code>opencompass/CompassJuder-1-1.5B-Instruct</code>	1.5	73.4	Generative
<code>OpenAssistant/oasst-rm-2.1-pythia-1.4b-epoch-2.5</code>	1.4	69.5	Seq. Classifier
<code>Finemath-RM-Llama-3.2-1B (ours)</code>	1.0	60.0	Seq. Classifier
<code>Qwen/Qwen1.5-0.5B-Chat</code>	0.5	53.8	DPO

Table 16: `RewardBench 2` leaderboard RMs in the 0.5–1.5B parameter range, sorted by AvgAcc.

Model	Params (B)	AvgAcc	Type
<code>Skywork/Skywork-Reward-V2-Llama-3.2-1B</code>	1.0	64.376	Seq. Classifier
<code>Skywork/Skywork-Reward-V2-Qwen3-0.6B</code>	0.6	61.250	Seq. Classifier
<code>Finemath-RM-Llama-3.2-1B (ours)</code>	1.0	33.2	Seq. Classifier
<code>OpenAssistant/oasst-rm-2.1-pythia-1.4b-epoch-2.5</code>	1.4	26.479	Seq. Classifier

Table 17: RewardBench leaderboard RMs in the 3-4B parameter range, sorted by AvgAcc.

Model	Params (B)	AvgAcc	Type
Ray2333/GRM-llama3.2-3B-rewardmodel-ft	3.0	90.9	Seq. Classifier
stabilityai/stablelm-zephyr-3b	3.0	74.0	DPO
Finemath-RM-Qwen-2.5-3B (ours)	3.0	70.0	Seq. Classifier
Finemath-RM-Llama-3.2-3B (ours)	3.0	65.6	Seq. Classifier
stabilityai/stable-code-instruct-3b	3.0	64.3	DPO
Qwen/Qwen1.5-4B-Chat	4.0	56.0	DPO
ContextualAI/archangel_sft-kto_pythia1-4b	4.0	55.9	DPO
ContextualAI/archangel_sft-dpo_pythia1-4b	4.0	52.1	DPO
weqweasdas/hh_rlhf_rm_open_llama_3b	3.0	48.4	Seq. Classifier

Table 18: RewardBench 2 leaderboard RMs in the 3-4B parameter range, sorted by AvgAcc.

Model	Params (B)	Avg Acc	Type
Skywork/Skywork-Reward-V2-Qwen3-4B	4.0	75.51	Seq. Classifier
Skywork/Skywork-Reward-V2-Llama-3.2-3B	3.0	74.665	Seq. Classifier
Schrieffer/Llama-SARM-4B	4.0	73.793	Seq. Classifier
Skywork/Skywork-Reward-V2-Qwen3-1.7B	1.7	68.176	Seq. Classifier
Skywork/Skywork-Reward-V2-Llama-3.2-1B	1.0	64.376	Seq. Classifier
Skywork/Skywork-Reward-V2-Qwen3-0.6B	0.6	61.25	Seq. Classifier
Ray2333/GRM-gemma2-2B-rewardmodel-ft	2.0	59.661	Seq. Classifier
Finemath-RM-Qwen-2.5-3B (ours)	3.0	46.2	Seq. Classifier
Finemath-RM-Llama-3.2-3B (ours)	3.0	36.2	Seq. Classifier
weqweasdas/RM-Gemma-2B	2.0	30.566	Seq. Classifier
OpenAssistant/oasst-rm-2.1-pythia-1.4b-epoch-2.5	1.4	26.479	Seq. Classifier
weqweasdas/hh_rlhf_rm_open_llama_3b	3.0	24.98	Seq. Classifier

Table 19: RewardBench leaderboard RMs of 7B parameter size, sorted by AvgAcc.

Model	Params (B)	AvgAcc	Type
Skywork/Skywork-VL-Reward-7B	7.0	90.070	Seq. Classifier
R-I-S-E/RISE-Judge-Qwen2.5-7B	7.0	88.191	Generative
internlm/internlm2-7b-reward	7.0	87.593	Seq. Classifier
ZiyiYe/Con-J-Qwen2-7B	7.0	87.120	Generative
opencompass/CompassJudgeter-1-7B-Instruct	7.0	83.167	Generative
CIR-AMS/BTRM_Qwen2_7b_0613	7.0	83.152	Seq. Classifier
openbmb/Eurus-RM-7b	7.0	82.823	Seq. Classifier
weqweasdas/RM-Mistral-7B	7.0	80.389	Seq. Classifier
Ahjeong/MMPO_Gemma_7b_gamma1.1_epoch3	7.0	77.444	DPO
NousResearch/Nous-Hermes-2-Mistral-7B-DPO	7.0	77.222	DPO
Ray2333/reward-model-Mistral-7B-instruct-Unified-Feedback	7.0	76.896	Seq. Classifier
0-hero/Matter-0.1-7B-boost-DPO-preview	7.0	76.831	DPO
Ahjeong/MMPO_Gemma_7b	7.0	76.810	DPO
HuggingFaceH4/zephyr-7b-alpha	7.0	76.470	DPO
HuggingFaceH4/zephyr-7b-beta	7.0	75.385	DPO

Continued on next page

Table 19 (continued)

Model	Params (B)	AvgAcc	Type
allenai/tulu-2-dpo-7b	7.0	75.163	DPO
0-hero/Matter-0.1-7B-DPO-preview	7.0	74.847	DPO
Finemath-RM-Qwen-2.5-7B (ours)	7.0	73.8	Seq. Classifier
prometheus-eval/prometheus-7b-v2.0	7.0	72.043	Generative
berkeley-nest/Starling-RM-7B-alpha	7.0	71.529	Seq. Classifier
ai2/tulu-2-7b-rm-v0-nectar-binarized-700k.json	7.0	71.275	Seq. Classifier
openbmb/Eurus-7b-kto	7.0	71.048	DPO
ai2/tulu-2-7b-rm-v0-nectar-binarized-3.8m-checkpoint-380k.json	7.0	70.584	Seq. Classifier
Qwen/Qwen1.5-7B-Chat	7.0	70.579	DPO
ai2/tulu-2-7b-rm-v0-nectar-binarized-3.8m-checkpoint-2660k.json	7.0	70.193	Seq. Classifier
ai2/tulu-2-7b-rm-v0-nectar-binarized-3.8m-checkpoint-3420k.json	7.0	70.079	Seq. Classifier
ai2/tulu-2-7b-rm-v0-nectar-binarized-3.8m-checkpoint-3.8m.json	7.0	70.037	Seq. Classifier
HuggingFaceH4/zephyr-7b-gemma-v0.1	7.0	69.562	DPO
weqweasdas/RM-Gemma-7B	7.0	69.542	Seq. Classifier
ai2/tulu-2-7b-rm-v0-nectar-binarized-3.8m-checkpoint-3040k.json	7.0	69.450	Seq. Classifier
ai2/tulu-2-7b-rm-v0-nectar-binarized-3.8m-checkpoint-1900k.json	7.0	69.242	Seq. Classifier
allenai/OLMo-7B-Instruct	7.0	69.216	DPO
weqweasdas/RM-Gemma-7B-4096	7.0	69.095	Seq. Classifier
ai2/tulu-2-7b-rm-v0-nectar-binarized-3.8m-checkpoint-760k.json	7.0	69.045	Seq. Classifier
ai2/tulu-2-7b-rm-v0-nectar-binarized-3.8m-checkpoint-2280k.json	7.0	68.954	Seq. Classifier
ai2/tulu-2-7b-rm-v0-nectar-binarized-3.8m-checkpoint-1140k.json	7.0	68.084	Seq. Classifier
ai2/tulu-2-7b-rm-v0-nectar-binarized.json	7.0	67.558	Seq. Classifier
RLHFlow/RewardModel-Mistral-7B-for-DPA-v1	7.0	67.038	Seq. Classifier
ai2/tulu-2-7b-rm-v0.json	7.0	66.546	Seq. Classifier
PKU-Alignment/beaver-7b-v2.0-reward	7.0	63.906	Seq. Classifier
IDEA-CCNL/Ziya-LLaMA-7B-Reward	7.0	63.681	Seq. Classifier
PKU-Alignment/beaver-7b-v2.0-cost	7.0	60.267	Seq. Classifier
ai2/llama-2-chat-7b-nectar-3.8m.json	7.0	58.427	Seq. Classifier
PKU-Alignment/beaver-7b-v1.0-cost	7.0	58.098	Seq. Classifier
ContextualAI/archangel_sft-kto_llama7b	7.0	53.649	DPO
ContextualAI/archangel_sft-dpo_llama7b	7.0	52.737	DPO
PKU-Alignment/beaver-7b-v1.0-reward	7.0	45.684	Seq. Classifier

Table 20: RewardBench 2 leaderboard RMs of 7B parameter size, sorted by AvgAcc.

Model	Params (B)	Avg Acc	Type
Skywork/Skywork-VL-Reward-7B	7.0	68.847	Seq. Classifier
weqwesasdas/RM-Mistral-7B	7.0	59.601	Seq. Classifier
openbmb/Eurus-RM-7b	7.0	58.057	Seq. Classifier
CIR-AMS/BTRM_Qwen2_7b_0613	7.0	57.363	Seq. Classifier
Finemath-RM-Qwen-2.5-7B (ours)	7.0	57.0	Seq. Classifier
internlm/internlm2-7b-reward	7.0	53.348	Seq. Classifier
weqwesasdas/RM-Gemma-7B	7.0	48.255	Seq. Classifier
PKU-Alignment/beaver-7b-v1.0-cost	7.0	33.322	Seq. Classifier
PKU-Alignment/beaver-7b-v2.0-cost	7.0	33.261	Seq. Classifier
PKU-Alignment/beaver-7b-v2.0-reward	7.0	25.435	Seq. Classifier
PKU-Alignment/beaver-7b-v1.0-reward	7.0	16.057	Seq. Classifier

I GRPO Objective and Actor-Training Details

I.1 GRPO Objective

For each prompt x , we sample K completions $\{y_i\}_{i=1}^K \sim \pi_\theta(\cdot | x)$, score them with $r_\phi(x, y_i)$, and compute group-relative advantages:

$$A_i = \frac{r_\phi(x, y_i) - \mu_x}{\sigma_x + \epsilon}, \quad (10)$$

where μ_x and σ_x are the mean and standard deviation of the K scores for x , and ϵ stabilizes the denominator. The actor minimizes a clipped PPO-style surrogate with reference-policy regularization:

$$\mathcal{L}_{\text{actor}} = \mathcal{L}_{\text{clip}}(\pi_\theta; \pi_{\theta_{\text{old}}}, A) + \lambda \mathbb{E}_t \left[\mathcal{D}_{\text{KL}}(\pi_\theta(\cdot | s_t) \parallel \pi_{\text{ref}}(\cdot | s_t)) \right]. \quad (11)$$

Here λ controls deviation from the fixed reference policy, limiting policy drift under an imperfect RM. In implementation, the reference term is an MSE-style penalty on token log-probability differences [46, 54].

I.2 Training Setup and Results

We train actors with GRPO using the open-source framework `verl` [49] and the hyperparameters in Table 21. We report the best mean@1 test accuracy reached within the five-epoch budget in Table 5, including the `beaver-7b-unified-reward` baseline.

Table 21: Actor training hyperparameters for GRPO experiments. λ denotes the coefficient of the reference regularization term in Eq. (11); rollout K is the number of sampled completions per prompt.

Task	Ep.	Optimizer	LR	λ KL	Rollout K	Max len.	Batch size	Eval metric
GSM8K	5	Adam betas 0.9, 0.95	10^{-6}	0.1	8	512	512	mean@1 test accuracy using <code>math-verify</code>
MATH	5	Adam betas 0.9, 0.95	10^{-6}	0.1	8	512	512	mean@1 test accuracy using <code>math-verify</code>

J Extended Results for Continuing Training on Curated Preference Dataset

Unless otherwise noted, all RMs are trained using the open source framework `LlamaFactory` [61] following the hyperparameters in Table 22. In Figure 12, we plot for different initialization the RewardBench 2 score of the trained RM over the 1 epoch training course on the curated preference dataset `RLHFMix1`. Results reported in Table 6 are the RewardBench 2 performance gain achieved at the end of the training.

Table 22: Hyperparameters for RM training with curated preference dataset `RLHFMix1`. Cutoff length indicates the token number limits for prompt and response concatenated sequence.

Dataset	Epochs	Optimizer	LR	Warmup ratio	Cutoff len.	Batch size
<code>RLHFMix1</code>	1	Adam betas 0.9, 0.95	$2e-6$	0.03	2048	512

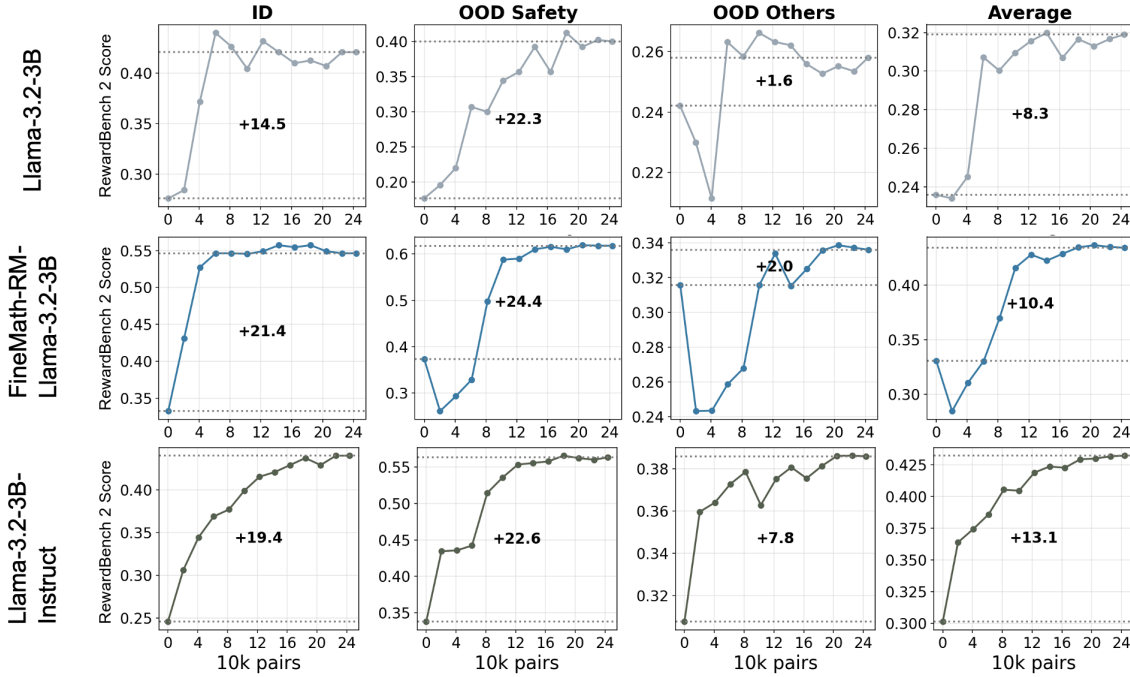


Figure 12: RewardBench 2 curve across the 1 epoch training course on the curated preference dataset `RLHFMix1` for 3 representative initializations `Llama-3.2-3B` base model, `FineMath-RM-Llama-3.2-3B` trained from our method, and `Llama-3.2-3B-Instruct`.

J.1 Label Efficiency of NTP Initialization

To measure how NTP initialization affects the amount of curated supervision needed for reward modeling, we compare the base Llama-3.2-3B checkpoint with its NTP-trained counterpart across six fractions of RLHFMix1. NTP initialization uses 11M tokens of *FineMath-4plus* before curated preference training. We report absolute overall RewardBench 2 scores, allowing direct comparison between the two initializations at each labeled-data budget.

Table 23: Overall RewardBench 2 scores after training on different fractions of RLHFMix1, comparing base and NTP initialization. Pair counts and scores are approximate; the full dataset contains 244,216 pairs.

Labeled fraction	Pairs	Base init.	NTP init.
1%	2.4K	23	32
5%	12K	23	29
10%	24K	24	29
25%	61K	31	33
50%	122K	32	44
100%	244K	32	44

Table 23 shows higher scores from NTP initialization at every evaluated budget. With only 1% of the labeled data (approximately 2.4K pairs), NTP initialization reaches approximately 32, compared with 23 for base initialization. At 50% (approximately 122K pairs), NTP initialization reaches approximately 44, matching its result with the full dataset. Base initialization reaches approximately 32 at both 50% and 100%, leaving a gap of about 12 points at these budgets.

Thus, NTP initialization reaches its full-data performance using half of the curated preference pairs, while exceeding the score attained by base initialization at any of the tested budgets. The benefit is not a uniform improvement as more labels are added: the NTP-initialized model’s score decreases from approximately 32 at 1% to 29 at 5–10%, before recovering and improving at larger budgets. Nevertheless, its advantage over base initialization persists throughout the sweep. These results support using NTP training on web text to improve the label efficiency of subsequent supervised reward modeling.

K Licenses for Existing Assets

Datasets.

- **FineMath-4plus** [2] — released by HuggingFaceFW under the ODC-By 1.0 license.
- **FineWeb-Edu** [44] — released by HuggingFaceFW under the ODC-By 1.0 license.
- **MegaMath-Web-Pro-Max** [56] — the OctoThinker dataset card lists ODC-By 1.0 and specifies non-commercial use and compliance with the original source licenses.
- **RLHFMix1** — a mixture of UltraFeedback [13] (MIT), HH-RLHF [3] (MIT), SHP [21] (CC BY 4.0), and Summarize from Feedback [53] (MIT).
- **Skywork-Reward-Preference-80K-v0.2** [35] — released under CC BY 4.0.
- **GSM8K** [12] — released by OpenAI under MIT.
- **MATH** [30] — released under MIT.
- **ToxiGen** [28] — released under MIT.
- **IFEval** [63] — released under Apache 2.0.

- **HumanEval+** [8, 37] — HumanEval released by OpenAI under MIT; the EvalPlus test suites released under Apache 2.0.
- **RewardBench** [33] and **RewardBench 2** [40] — released under Apache 2.0.

Models.

- **Llama 3 family** (Llama-3.2-1B, Llama-3.2-3B, Llama-3.1-8B and Instruct variants) [41] — released under the Llama 3 Community License.
- **Qwen2.5 family** (Qwen2.5-3B-Instruct, Qwen2.5-7B-Instruct) [45] — released under Apache 2.0.
- **Skywork-Reward-V2** models [36] — released under Apache 2.0.
- **PKU-Alignment/beaver-7b-unified-reward** [14, 31] — released under a non-commercial license.

Frameworks and Tools.

- **verl** [49] — released under Apache 2.0.
- **LlamaFactory** [61] — released under Apache 2.0.
- **lm-evaluation-harness** [24] — released under MIT.
- **math-verify** — released under Apache 2.0.

L Compute Resources

All experiments are run on NVIDIA A100 or H100 80GB GPUs. The approximate compute cost per experimental type is summarized below.

- **RM training.** Each run uses 1, 2, or 4 H100 GPUs and takes 1–2 days depending on the backbone size and training budget.
- **CPT training.** The continued-pretraining control uses 4 H100 GPUs, processes 11M tokens in one epoch, and takes less than one hour.
- **Best-of- N (BoN) evaluation.** Each evaluation run uses 1 H100 GPU and takes a few hours.
- **Actor training.** Each run uses 2 H100 GPUs and takes approximately half a day.
- **RM mid-training (curated preference dataset).** Each run uses 1 H100 GPU and takes a few hours.